REFERENZ

Durchführung eines Tests

Das Testformat im Wortlaut der Quelle: Trommeldatei, Tastenfolge, Prüfungen, Geräte, Protokoll

Dieses PDF gibt die Quelle im Wortlaut wieder; Listen, Tabellen und Codeblöcke entsprechen der Quelle, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

Quelle	C8205-Interpreter-Durchführung_eines_Tests.md	0.95-014
Stand	25.10.2026	
Umfang	36 Seiten, A4 hoch	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

C 8205 – Durchführung eines Tests

Quelle: C8205-Interpreter-Durchführung_eines_Tests.md · im Wortlaut der Quelle

Inhaltsverzeichnis

1. Einordnung und Testvoraussetzungen . . . 4

1a. Zwei Ausführende: Python-Interpreter und Claude-Interpreter . . . 4

1b. Test starten: Claude-Interpreter (ohne Python) und Python-Interpreter (Ergänzung 06.10.2026) . . 4

1b.1 Claude-Interpreter (ohne Python) . . . 4

1b.2 Python-Interpreter . . . 5

2. Schritt 1 jedes Tests: die Trommeldatei . . . 6

2a. Testauftrag aus einem Assemblerlauf (Ergänzung 01.10.2026) . . . 8

2b. Geräte-Zeilen --leser1, --leser2, --stanzer1, --sw-ausgabe und --k0 in der Testdatei (Ergänzung 03.10.2026, erweitert 04.10.2026) . . . 9

2c. Die Trommel als persistente Datei (Ergänzung 07.10.2026) . . . 9

2d. Unterbrechung des Programmlaufs durch #INTERRUPT (Ergänzung 08.10.2026) . . . 10

2e. Das Makro #STARTE für den Ansprung eines Speicherplatzes (Ergänzung 09.10.2026) . . . 12

2f. Tests der BIS-Bausteine: das Minimalprogramm läuft zuerst (Ergänzung 08.10.2026) . . . 13

2g. Die Testmarke #TROMMEL=LEER, #TROMMEL=KEEP und #TROMMEL=MP (Ergänzung 13.10.2026, KEEP 24.10.2026) . . . 15

2h. Beispiel 1: Ein Programm über Lochstreifen auf die Trommel laden und in TEST und GRAFIK ausführen (Ergänzung 20.10.2026) . . . 16

2h.1 Schritt für Schritt . . . 16

2h.2 Hinweise . . . 17

2i. Beispiel 2: Ein Assemblerprogramm anlegen, assemblieren und im GRAFIK-Interpreter ausführen (Ergänzung 20.10.2026) . . . 18

2i.1 Schritt für Schritt . . . 18

2i.2 Unterschied zu Beispiel 1 . . . 19

2j. Beispiel 3: Ein absolut assembliertes Programm als binären Lochstreifen mit Hilfe des KEP laden (Ergänzung 22.10.2026) . . . 19

2j.1 Schritt für Schritt . . . 19

2j.2 Hinweise . . . 20

2k. Lademethoden und ihr Angang am Assemblerquelltext (Ergänzung 24.10.2026) . . . 20

2k.1 Übersicht . . . 20

2k.2 Die drei Quelltextfassungen des Druckprogramms . . . 21

2k.3 Der BIS-Weg Schritt für Schritt (bis_test.sh) . . . 21

2k.4 Fallen am Quelltext . . . 21

2k.5 Die drei Skripte in Testausfuehrungen/ . . . 22

3. Schritt 2 jedes Tests: die Tastenfolge zur Definition des Ausgangszustands . . . 22

3.1 Beispiel A — Ansprung des Speicherplatzes 0000 (direkt) . . . 22

3.2 Beispiel B — Ansprung eines beliebigen Speicherplatzes (hier: 0004) . . . 22

3.3 Festlegungen zu den Tasten M, Ü und E (Stand 29.09.2026) . . . 23

3a. Schritt 3 jedes Tests (optional): Ergebnisprüfung (Prüfungen:) . . . 24

3a.1 Aufbau . . . 24

3a.2 Auswertung . . . 25

3a.3 Ausgabe . . . 25

3a.4 Formale Gültigkeitsprüfung . . . 25

3a.5 Erwartetes Ergebnis „falsch“: Prüfungen: False (Ergänzung 10.10.2026) . . . 26

3b. Dateiprüfungen für Stanzer und Schreibwerk (--pruefe-stanzer1, --pruefe-sw-ausgabe) . . . 26

3b.1 Soll-Datei und Ist-Datei . . . 27

3b.2 Auswertung . . . 27

3b.3 Ausgabe . . . 27

3b.4 Formale Gültigkeitsprüfung . . . 27

4. Weitere typische Tastensequenzen (zur Illustration, nicht Teil des Test-Starts) . . . 28

4.1 Beseitigen der Betriebszustände US, BS, FÜ und FE . . . 28
4.2 Transport des Akkumulatorinhaltes nach Speicherplatz 7777 . . . 29
4.3 Transport des Befehlszählerinhaltes nach dem Akkumulator . . . 30

5. Formale Gültigkeitsprüfung vor Testbeginn . . . 30

6. Interaktive Zeicheneingabe und -ausgabe während der interpretativen Abarbeitung . . . 31
6.0 Kanalanwahl: Bedienpult-Tasten und Anwahlbefehl . . . 31
6.1 Kanal K0: Eingabe (Rechnerverhalten und Eingabequelle sind unabhängig) . . . 31
6.2 Kanäle K1/K2 (Lochstreifenleser): Eingabe über virtuelle Lochstreifen-Dateien . . . 33
6.3 Ausgabebefehle . . . 33

7. Diagnoseausgabe bei Befehl 164 (Leerbefehl) mit Testmarken #DOK/#SPEI . . . 35
7.1 Auslösebedingung . . . 35
7.2 Ablauf . . . 35

8. Protokolldatei je Test . . . 35
8.1 Inhalt . . . 35
8.2 Bildschirmausgabe der Zustände des Knotenautomaten . . . 36
8.3 Bereitstellung am Testende . . . 36

9. Ausblick . . . 36

Version: 0.95-014

1. Einordnung und Testvoraussetzungen

Das Gesamtdokument (mit all seinen Teilen) beschreibt zwei unterschiedliche Aspekte:

1. Die Hardware und die in der Elektronik angelegten Beziehungen (Knotenautomat).
2. Davon abweichend/ergänzend die Annahmen für eine interpretative Abarbeitung eines Befehls.

Punkt 2 beschreibt damit die Voraussetzung jedes Tests. Insbesondere basiert er auf der Annahme, dass alle Zeiten = 0 sind, d. h. zeitliche Abhängigkeiten werden zwar in ihrer Reihenfolge betrachtet, führen aber nicht zu "Wartezeiten".

Jeder Test nimmt an, dass der Rechner eingeschaltet ist.

Der Vor-Zustand vom Einschalten bis zum Beginn der Interpretation der Befehlsschleife am Konnektor B wird definiert durch:

- die Interpretation der betätigten Tasten und Schalter,
- den Trommelinhalt: den Inhalt der persistenten Trommel, nachdem die Zeilen des Tests auf sie geschrieben wurden (Abschnitte 2 und 2c; mit der Testmarke `#TROMMEL=` zuvor geleert oder mit einem Abbild belegt, Abschnitt 2g), sowie
- optional die Anfangswerte von Akkumulator und Befehlszähler, die die Testmarken `#AC=` und `#BZ=` vorgeben (Abschnitt 2, Ergänzung 08.10.2026).

1a. Zwei Ausführende: Python-Interpreter und Claude-Interpreter

Ein Test nach diesem Format kann auf zwei Arten durchgeführt werden:

- **Python-Interpreter** (`c8205_interp.py`): führt den Knotengraphen deterministisch und automatisiert aus. Schneller, reproduzierbar (bei definiertem Trommelstand, Abschnitt 2c), aber ohne Rückfragemöglichkeit bei unklaren Stellen.
- **Claude-Interpreter**: Eine KI erhält das komplette Regelwerk als Kontext sowie eine Testdatei (hochgeladen oder im Prompt) und arbeitet den Knotengraphen manuell, Knoten für Knoten, ab. Langsamer und nicht deterministisch reproduzierbar (abhängig von Modell und Sitzung), erlaubt dafür aber Rückfragen und Erläuterungen während der Abarbeitung.

Rolle im Normalbetrieb: Der Python-Interpreter ist die Standardmethode für Testläufe. Der Claude-Interpreter dient als unabhängige Gegenprobe: Weichen Python-Ergebnis und Claude-Ergebnis bei einem Test voneinander ab, liegt ein Fehler in einem der beiden vor (oder eine unklare Stelle im Regelwerk selbst). Außerdem wird er eingesetzt, wenn im Einzelfall Rückfragen zu einer Ableitung oder Unklarheit gestellt werden sollen.

Rolle bei künftigen Erweiterungen des Regelwerks: Neue oder geänderte Befehle/Anweisungen werden zunächst mit dem Claude-Interpreter durchgespielt, bevor sie in `c8205_interp.py` übernommen werden.

Einschränkung: Der Claude-Interpreter ist nicht deterministisch reproduzierbar. Bei unklaren Regelwerk-Stellen kann die ausführende KI Rückfragen stellen, statt eigenmächtig zu entscheiden (vgl. Abschnitt 5, "laut scheitern"-Prinzip) – abhängig vom verwendeten Modell mit unterschiedlich vielen Rückfragen. Diese Festlegung gilt modellneutral für jede interpretierende KI, nicht nur für Claude.

Den Start eines Tests auf beiden Wegen beschreibt Abschnitt 1b.

1b. Test starten: Claude-Interpreter (ohne Python) und Python-Interpreter (Ergänzung 06.10.2026)

Dieser Abschnitt fasst zusammen, wie ein Test auf den beiden Wegen aus 1a gestartet wird. Er legt nichts Neues fest, sondern verweist auf die Abschnitte, in denen die Einzelheiten stehen. Die Startanweisung in 1b.1 ist ein Vorschlag und nicht Teil des Regelwerks.

1b.1 Claude-Interpreter (ohne Python)

1. **Unterhaltung beginnen**, in der Dateien hochgeladen werden können. Die KI führt den Knotengraphen von Hand aus; es wird kein Programm gestartet.

2. **Regelwerk hochladen** (Ordner `markdown-dateien/`):
   - immer: `C8205-Interpreter-Eingänge_der_Knoten.md`, `C8205-Interpreter-Beschreibung_der_Knoten.md` (Variablentabelle T2), `C8205-Interpreter-Befehlsliste.md`, `C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md`, `C8205-Interpreter-Bedienpult_mit_Schaltern.md` und diese Datei;
   - je nach Test: `C8205-Interpreter-Magnettrommel.md` (bei `#SPUR`/`#SPEI`), `C8205-Interpreter-Kodetabellen.md` und `C8205-Interpreter-Lochstreifenleser.md` (bei Ein- und Ausgabe, Abschnitt 6) sowie `C8205-Interpreter-Überblick_und_Gesamtaufbau.md` (optional, für Rückfragen zu den Phasen);
   - nicht nötig: `C8205-Interpreter-Python_Interpreter.md`, `C8205-Interpreter-PDF.md`, `C8205-Projektstruktur.md`, `C8205-Interpreter-Projektueberblick.md`, `C8205-Versionierung.md`, `C8205-Interpreter-Lochstreifen-Hilfsprogramme.md` und die Assembler-Dokumentation;
   - `baz-wirkungstabelle.md` ist optional und nur dann zu verwenden, wenn ihre Einträge zu den aktuellen Fassungen von Eingänge_der_Knoten, Beschreibung_der_Knoten und Befehlsliste passen; bei geänderten Quelldateien sind die betroffenen Einträge neu herzuleiten (Kopf der Datei).

   Wer sich die Auswahl sparen will, lädt alle `.md`-Dateien außer den unter „nicht nötig" genannten hoch. Das kostet Kontext, den lange Protokolle brauchen.
3. **Testdatei bereitstellen** (Format: Abschnitte 2 bis 3b), hochgeladen oder im Prompt. Geräte-Zeilen (Abschnitt 2b) gelten beim Claude-Interpreter als virtuelle Zuordnung. Lochstreifendateien für die Leser werden bereitgehalten; die KI fordert sie an, wenn ein Eingabebefehl sie braucht (Abschnitte 6.1 und 6.2). Eine Trommeldatei braucht die KI nicht: Zellen, die der Test nicht nennt, gelten als 0 (Abschnitt 2c).
4. **Startanweisung** in den Chat schreiben, zum Beispiel:

```
Führe den hochgeladenen Test <Testname> als Claude-Interpreter aus, ohne Python oder ein anderes Programm.
Regelwerk sind die hochgeladenen .md-Dateien. Prüfe zuerst die Testdatei formal.
Arbeite dann den Knotengraphen (Eingänge_der_Knoten, Variablentabelle T2 in Beschreibung_der_Knoten)
Knoten für Knoten ab und führe das Protokoll in der Stufe, die #PROT STUFE vorgibt.
Bei jeder Unklarheit, jedem Widerspruch und jeder fehlenden Festlegung: anhalten und rückfragen,
nichts annehmen. Gib am Ende die Endzeile und die Ergebnisprüfung (Test wahr / Test falsch) aus.
```

5. **Ergebnis lesen**: Endzeile, Ergebnisprüfung (*Test wahr* oder *Test falsch*) und Protokoll (Abschnitt 8). Rückkehrcodes gibt es nur beim Python-Aufruf (`C8205-Interpreter-Python_Interpreter.md`). Stanzer- und Schreibwerk-Ausgabe stellt die KI am Testende als Datei zum Download bereit (Abschnitte 6.3 und 8.3). Weicht das Ergebnis vom Python-Lauf ab, gilt 1a: Es liegt ein Fehler in einem der beiden Wege vor oder eine unklare Stelle im Regelwerk.

1b.2 Python-Interpreter

1. **Voraussetzung:** Python 3 (nur Standardbibliothek). Im Projektordner liegen `c8205_interp.py` und daneben `markdown-dateien/`; der Interpreter liest daraus `C8205-Interpreter-Eingänge_der_Knoten.md` und `C8205-Interpreter-Beschreibung_der_Knoten.md` (anderer Ort: Schalter `--mdverz`). Außerdem muss die Trommeldatei `Trommel/trommel.txt` vorhanden sein (leer erzeugen: `python3 testscripte/erzeuge_leere_trommel.py`; anderer Ort: Schalter `--trommel`; Abschnitt 2c).
2. **Testdatei ablegen** in `Tests/` (die Endung ist frei; der Assembler erzeugt `.test`). Geräte werden per Schalter oder Zeile zugeordnet (Abschnitt 2b); Dateien dafür liegen üblicherweise unter `Test-virtuelleGeraete/`.
3. **Starten**, direkt oder mit dem Wrapper, der das Protokoll nach `Test-Protokolle/` schreibt:

```
python3 c8205_interp.py --MODUS=TEST Tests/0015-030.txt
./c8205_interpreter.sh 0015-030.txt        # Linux/macOS, Protokoll: Test-Protokolle/0015-030.log
c8205_interpreter.bat 0015-030.txt         # Windows
testscripte/compute_all_tests.sh           # alle Testdefinitionen in Tests/
```

   Die Trommel behält ihren Inhalt von Test zu Test und von Aufruf zu Aufruf. Für einen Lauf auf leerer Trommel vorher `python3 testscripte/erzeuge_leere_trommel.py` aufrufen (Abschnitt 2c); die Hilfsskripte liegen im Ordner `testscripte/`.

4. **Ergebnis lesen**: Endzeile (`Ende: …`), Ergebnisprüfung (*Test wahr* oder *Test falsch*) und Rückkehrcode (0 wahr, 4 ohne Prüfung, 6 falsch, 8 Laufzeitfehler, 16 Vorab-Prüfung negativ oder Trommeldatei fehlerhaft, 2 ungültiger Aufruf; Code 6 bei einem Test mit `Prüfungen: False` werten die Skripte als wahr, Abschnitt 3a.5). Knotenzeilen erscheinen im Protokoll mit `--knotentest` und `#PROT STUFE=2` (Abschnitt 8). Alle Schalter, Geräte und Fehlermeldungen: `C8205-Interpreter-Python_Interpreter.md`.

GRAFIK: Tests dieses Dokuments laufen im Modus TEST (`--MODUS=TEST`). Die Prüfung des Modus GRAFIK (Logiktests, Tk-Tests, Handläufe, Hörprüfung) steht in `C8205-Interpreter-GRAFIK_Pruefverfahren.md`.

2. Schritt 1 jedes Tests: die Trommeldatei

Der Trommelinhalt, den ein Test braucht, wird, zum Zwecke der Interpretation, vor Beginn jedes Tests als .txt-Datei mit festem Aufbau übermittelt (Abschnitt `Trommeldatei:` der Testdatei). Das ist der erste Schritt jedes Tests.

Der Trommelinhalt der Testdatei ist nur ein Ausschnitt (adreßweise) der Gesamttrommel. Er wird vor dem Test auf die Trommel **geschrieben** und überlagert dort die vorhandenen Werte; Zellen, die er nicht nennt, behalten ihren bisherigen Inhalt (Abschnitt 2c: Die Trommel ist eine persistente Datei).

Zeilenformat:

```
Adresse: Wert [# Bemerkung]
```

- Adresse genügt der Definition von ADR (bzw. a): 4 Oktalstellen (12 Bit).
- Wert genügt der Definition Maschinenwort: 11 Oktalstellen (33 Bit), mit Vornullen ergänzt.
- Alle Zahlen werden als Oktalzahlen dargestellt.
- Leerzeilen sind möglich und werden übergangen.
- Kommentarzeilen beginnen mit `#`.
- Ein Kommentar beginnt mit `#` und reicht bis zum Zeilenende. Er kann entweder als eigene Zeile stehen oder – durch mindestens ein Leerzeichen getrennt – an eine Adresse:Wert-Zeile angehängt werden; in letzterem Fall gilt für das Parsen alles ab dem `#` bis Zeilenende als Kommentar und wird ignoriert.

Beispiel:

```
# Beginn Programmbefehle
0000: 00003000211
0001: 00003001210
0002: 00000000020
# Beginn Datenbereich
3000: 00000000100 # Startwert der Schleife
3001: 00000000045
```

Testmarken `#DOK-xxxx` und `#SPEI-xxxx` (von der Hardware abweichend, nur für die Testdurchführung):

Zusätzlich zu den oben beschriebenen Adresse:Wert-Zeilen kann die Trommeldatei Anweisungszeilen der Form

```
#DOK-xxxx <Text>
```

enthalten. `xxxx` ist dabei eine Oktalzahl; `<Text>` ist ein freier, dem Test zugeordneter Kommentartext. Diese Zeilen sind, obwohl sie wie die übrigen Kommentarzeilen mit `#` beginnen, keine gewöhnlichen, folgenlosen Kommentare, sondern Testmarken mit eigener Wirkung bei der Interpretation (s. Abschnitt 7).

#DOK-xxxx darf mehrfach mit derselben Nummer xxxx vorkommen (Ausgabe aller zugehörigen Texte in Dateireihenfolge). #SPEI-xxxx darf mehrfach mit derselben Nummer xxxx vorkommen und wird – falls vorhanden – im Anschluss an die Textzeile(n) gleicher Nummer ausgegeben.

```
#SPEI-xxxx ssss-eeee
```

gegeben werden. Sie definiert für dasselbe `xxxx` einen Speicherbereich des Trommelspeichers, `ssss` als Startadresse und `eeee` als Endadresse, beide oktal.

(Hinweis: Die Vorgabe zur Testmarke `#SPEI-xxxx` verlangt wörtlich `ssss>=eeee`; da `ssss` als Start- und `eeee` als Endadresse benannt sind, ist damit vermutlich `ssss<=eeee` gemeint — vgl. den analog behandelten mutmaßlichen Tippfehler in Abschnitt 3.2. Festzuhalten ist in jedem Fall die Forderung: Vor Beginn des Tests ist zu prüfen, dass `ssss` und `eeee` eine gültige Adressspanne bilden; ein Test mit einer nicht gültigen Spanne darf nicht gestartet werden.)

Ergänzung 29.09.2026 – Testmarke `#SPUR-xxxx aaaa`:

```
#SPUR-xxxx aaaa
```

`aaaa` ist eine Oktaladresse. Die Testmarke bezieht sich, anders als `#SPEI-xxxx`, nicht auf eine Adressspanne, sondern auf die **Spur (Bahn)**, in der `aaaa` liegt (Bahn-Nr. = `aaaa` ohne die unteren 5 Bit, s. Datei "C8205-Interpreter-Magnettrommel.md", Abschnitt Speicherorganisation); welcher der 32 Sektoren von `aaaa`

konkret adressiert wird, spielt dabei keine Rolle. Ausgegeben werden alle 32 Zellen dieser Spur, und zwar in der **physischen Zugriffsreihenfolge** der Trommel (nicht in aufsteigender logischer Adresse), wie in der Datei "C8205-Interpreter-Magnettrommel.md", Abschnitt "Speichernumerierung / Interleaving" tabelliert. `#SPUR-xxxx` darf wie `#SPEI-xxxx` mehrfach mit derselben Nummer xxxx vorkommen und wird, falls vorhanden, im Anschluss an eine etwaige `#SPEI-xxxx`-Ausgabe angehängt (Reihenfolge: erst alle `#SPEI-xxxx`, danach alle `#SPUR-xxxx`, jeweils in Dateireihenfolge). Das Zeilenformat je Zelle ist dasselbe wie bei `#SPEI-xxxx` ("Adresse: Wert").

Diese drei Testmarken sind eine von der realen Hardware abweichende Ergänzung, die ausschließlich der besseren Durchführung von Tests mit dem Interpreter dient; sie haben keine Entsprechung in der physischen Elektronik des C 8205.

Testmarke `#PROT STUFE=n`, n ∈ {0,1,2} (Protokollierungsstufe):

Zusätzlich zu `#DOK-xxxx` und `#SPEI-xxxx` kann die Trommeldatei eine Anweisungszeile

```
#PROT STUFE=n
```

enthalten. Anders als `#DOK-xxxx` und `#SPEI-xxxx` bezieht sie sich nicht auf eine einzelne Adresse, sondern gilt test-weit; sie legt fest, wie ausführlich der Durchlauf durch den Knotenautomaten (Abschnitt 8) protokolliert wird — nützlich vor allem bei Programmen mit vielen Durchläufen (z. B. Schleifen über Gruppen-/Wiederholungsbefehle), bei denen ein Protokoll jedes einzelnen durchlaufenen Knotens unübersichtlich lang würde.

- **STUFE=0:** Es wird kein ausführliches Protokoll des Knotenautomaten-Durchlaufs erstellt, weder auf dem Bildschirm noch in einer Datei. Die Diagnoseausgaben zu Befehl 164 (Abschnitt 7) bleiben davon unberührt: sie erscheinen unabhängig von der Protokollierungsstufe stets auf dem Bildschirm.
- **STUFE=1:** Zusätzlich zu den stets erscheinenden Diagnoseausgaben (Abschnitt 7) wird das ausführliche Protokoll des Knotenautomaten-Durchlaufs auf dem Bildschirm ausgegeben (s. Abschnitt 8.2). Es entsteht dabei keine Protokolldatei.
- **STUFE=2:** Zusätzlich zu den stets erscheinenden Diagnoseausgaben (Abschnitt 7) wird das ausführliche Protokoll des Knotenautomaten-Durchlaufs statt auf dem Bildschirm in die Protokolldatei ("protokoll.txt", s. Abschnitt 8) geschrieben, die am Testende zum Download bereitgestellt wird.

Die Marke darf innerhalb eines Tests höchstens einmal vorkommen. Fehlt sie, gilt STUFE=1 als Standardwert (entspricht der bislang unbedingt geltenden Protokollierung vor Einführung dieser Marke).

Wie `#DOK-xxxx` und `#SPEI-xxxx` ist auch `#PROT STUFE=n` eine von der realen Hardware abweichende, zusätzliche Definition, die ausschließlich der besseren Durchführung von Tests mit dem Interpreter dient und keine Entsprechung in der physischen Elektronik des C 8205 hat.

Ergänzung 08.10.2026 – Testmarken `#AC=` und `#BZ=` (Anfangswerte der Register):

```
#AC=<Oktalzahl mit 11 Ziffern>
#BZ=<Oktalzahl mit 4 Ziffern>
```

Beide Marken sind optional und voneinander unabhängig (es darf nur eine, beide oder keine angegeben sein). Sie gelten test-weit, stehen wie `#PROT STUFE=n` im Abschnitt `Trommeldatei:` (also vor der Tastenfolge) und dürfen jeweils **höchstens einmal** vorkommen. Zwischen `=` und dem Wert dürfen Leerzeichen stehen; ein Kommentar `# …` hinter der Marke ist erlaubt.

- `#AC=` gibt den Anfangswert des Akkumulators vor: 11 Oktalziffern (33 Bit, die Maschinenwortbreite wie bei einem Trommelwert nach Abschnitt 2).
- `#BZ=` gibt den Anfangswert des Befehlszählers vor: 4 Oktalziffern (12 Bit, wie eine Adresse nach Abschnitt 2).
- **Zeitpunkt:** Die Register werden als Anfangszustand des Tests **vor der ersten Taste und vor jedem Befehl** gesetzt. Die Tastenfolge (Abschnitt 3) wirkt danach auf diesen Zustand.
- **Fehlt eine Marke,** bleibt alles wie bisher: das betreffende Register startet mit 0 (`AC` = 00000000000, `BZ` = 0000).
- **Tasten danach wirken unverändert wie bisher:** Eine in der Tastenfolge abgearbeitete Taste `G` verhält sich wie in Abschnitt 3.1 beschrieben, auch wenn `#BZ=` angegeben ist: sie setzt weder BZ noch AC, sondern stellt das Befehlsregister auf 0000 000 (unbedingter Sprung nach 0000). Erst die Ausführung dieses Befehls (durch `R` oder durch `E` ohne `M`) setzt BZ := 0000; folgt dagegen `M+ E … E M-`, kommt es nicht zu diesem Sprung. Der Anfangswert `#BZ=` bleibt also nach `G` erhalten, bis dieser Sprung ausgeführt wird, und wirkt sonst z. B. bei der Tastenfolge `H M-gedrückt-halten E M-loslassen R` ohne `G`, oder wenn der Test, wie in 4.2 und 4.3 der Originaldokumentation, nach `G` mit dem Anfangs-AC weiterarbeitet (`H G Ü E` transportiert den vorgegebenen AC nach 7777). `#AC=` bleibt von `G` ohnehin unberührt.

- **Weitere Tasten:** `E` unter gehaltenem `M` (3.3) rechnet vom aktuellen BZ aus weiter: `BZ := BZ + 1`, und der AC wird dabei wie beschrieben überschrieben.
- Sonst bleiben `AR`, `OR`, Flags und die übrigen Register unverändert (alle 0).

Beispiel:

```
#PROT STUFE=2
#AC=00000000100
0000: 00003000210 # Einzel-Addition ohne Löschen: AC := AC + <3000>
0001: 00000000020 # Stopp
3000: 00000000045
```

Tastenfolge `H G R`: Der Test beginnt mit AC = 100; nach der Addition steht im AC der Wert 145 (`BZ=0002`).

Auch `#AC=` und `#BZ=` sind eine von der realen Hardware abweichende, zusätzliche Definition, die ausschließlich der besseren Durchführung von Tests mit dem Interpreter dient (Testbeispiele: `Tests/ac-001_AC_vorbelegt.txt`, `bz-001_BZ_vorbelegt_G.txt`, `acbz-001_AC_und_BZ_vorbelegt.txt`, `acbz-002_BZ_vorbelegt_M_E.txt`).

2a. Testauftrag aus einem Assemblerlauf (Ergänzung 01.10.2026)

Eine Testdatei (Testauftrag) lässt sich von Hand schreiben, ist wegen der oktalen Adressen aber mühsam. Für Programme in Assemblersprache übernimmt das der Assembler `c82ass` (Projektstruktur, Abschnitt 3): Er wandelt ein mit symbolischen Adressen geschriebenes Programm in Oktalbefehle um. Von den dabei erzeugten Dateien wird nur `<name>.test` benötigt. Sie enthält die Trommeldatei im Zeilenformat von Abschnitt 2, jede Zeile mit der zugehörigen Quelltextzeile als Bemerkung, sowie Platzhalter-Definitionen für Testmarken und Tastenfolge und ggf. den Leerbefehl 164, die für den jeweiligen Fall ergänzt bzw. erweitert werden müssen:

```
2007: 00002006241 # beginn: 241 i ; I:=0
2010: 00002003210 # schleife: 210 azv ; ac:=(azv+i)
```

Vorgehen:

1. Im Assembler-Quelltext an der Stelle, an der ausgewertet werden soll, einen Leerbefehl 164 mit einer vierstelligen Debug-Nummer als Adresse vorsehen (Abschnitt 7), z. B. `halt: 164 0001`. Die Nummer muss mit den Testmarken `#DOK-`/`#SPEI-`/`#SPUR-` des Testauftrags übereinstimmen.
2. `c82ass.sh -p <Projekt> -s <name>` aufrufen.
3. `<name>.test` als Testauftrag `Tests/asm_NNNN_<name>.txt` ablegen und die darin enthaltenen Platzhalter-Definitionen für den Fall ergänzen bzw. erweitern: die Testmarken (`#DOK`, `#SPEI`, `#PROT STUFE=n`, bei Bedarf `#AC=`/`#BZ=`) mit den Debug-Nummern aus Schritt 1, die Tastenfolge (im Beispiel `H G R`: Start bei Adresse 0000, dort steht ein Sprung zum Programmanfang), ggf. den Leerbefehl 164 und optional den Abschnitt `Prüfungen:` (Abschnitt 3a).

Von Hand angepasst werden also nur die Platzhalter für die Debug-Anweisungen (Testmarken) und die Startbedingungen (Tastenfolge) sowie ggf. der Leerbefehl 164 und die Prüfungen. Der Code selbst stammt unverändert aus dem Assemblerlauf.

Beispiel (gekürzt; Quelltext `Assembler/Projekt_1/bsp_5.16/bsp_5_16.asm`, Test `Tests/asm_0001_bsp_5_16.txt`):

```
Trommeldatei:
#DOK-0001 Test eines mit Assembler assemblierten Programmes
#SPEI-0001 2027-2033
#SPEI-0001 2100-2104
#PROT STUFE=2

0000: 00002000000 # 000 go
2000: 00002007000 # go: 000 beginn
...                                  (weitere Zeilen unverändert aus bsp_5_16.test)
2025: 00000001164 # halt: 164 0001 ;DEBUG Nr. 0001 aktivieren
2026: 00000000020 # 020 0 ;halt
...

Tastenfolge:
H G R
```

2b. Geräte-Zeilen `--leser1`, `--leser2`, `--stanzer1`, `--sw-ausgabe` und `--k0` in der Testdatei (Ergänzung 03.10.2026, erweitert 04.10.2026)

Die Zuordnung der Schreibwerk-Ausgabe (6.3.1) und der K0-Eingabequelle (6.1.2) kann nicht nur als Schalter des Python-Interpreters, sondern auch **in der Testdatei selbst** angegeben werden. Der Testauftrag bringt damit seine Ein-/Ausgabe mit. Fünf Zeilenformen stehen zur Verfügung (eingeschlossen die Lochstreifenleser, 6.2.1, und der Stanzer, 6.3.2):

```
--leser1 PFAD
--leser2 PFAD
--stanzer1 PFAD
--sw-ausgabe PFAD
--k0 datei=PFAD
```

- **Stellung:** an beliebiger Stelle der Testdatei, üblicherweise bei den Testmarken (`#DOK`, `#SPEI`, `#PROT`) im Abschnitt `Trommeldatei:`. Je Zeile ein Schalter; ein Kommentar `# …` am Zeilenende wird ignoriert.
- **Höchstens einmal:** jede dieser Zeilen darf in einer Testdatei höchstens einmal vorkommen.
- **Wert:** wie beim gleichnamigen Schalter. `--sw-ausgabe PFAD`: `PFAD` ist das Verzeichnis (oder der Dateiname), in dem "sw-ausgabe.txt" angelegt wird (6.3.1). `--k0 datei=PFAD`: Datei oder Verzeichnis der K0-Eingabe (6.1.2).
- **Vorrang:** Ist derselbe Schalter zusätzlich auf der Kommandozeile angegeben, gilt die Kommandozeile; die Zeile der Testdatei wird dann übergangen (wie bei den Dateiprüfungen nach 3b).
- **Relative Pfade** gelten wie auf der Kommandozeile, also vom aktuellen Verzeichnis des Aufrufs aus, nicht vom Verzeichnis der Testdatei.
- **Nicht als Zeile:** `--SW-TERMINAL`**.** Der Kommandozeilenschalter `--SW-TERMINAL` (6.3.1) gehört nicht in die Testanforderung. Er wird erst bei der Durchführung des Tests ausgewertet und steht deshalb nur im Aufruf des Interpreters; eine Zeile `--SW-TERMINAL` in der Testdatei ist ein Formfehler der Vorab-Prüfung (Abschnitt 5), der Test startet nicht.
- **Lochstreifen und Stanzer:** `--leser1 PFAD` und `--leser2 PFAD` nennen die Streifendatei des Lesers 1 bzw. 2 (6.2.1); ein Verzeichnis ist möglich (alle Dateien darin werden in Namensreihenfolge als Streifen gelesen, 6.2.2), für einen Test ist aber eine einzelne Datei sicherer. `--stanzer1 PFAD` nennt die Ausgabedatei bzw. das Ausgabeverzeichnis des Stanzers (6.3.2). Damit bringt ein Test seine Lochstreifen selbst mit und lässt sich ohne weitere Schalter starten.
- **Claude-Interpreter:** Die Zeile gilt als virtuelle Zuordnung bei der Testdefinition.
- **Formale Prüfung (Abschnitt 5):** Eine Zeile ohne Wert, eine doppelt angegebene Zeile und `--k0` ohne `datei=` sind Formfehler; ebenso ein nicht vorhandener `--k0-`, `--leser1-` oder `--leser2`-Pfad. Der Test startet dann nicht.

Beispiel (Unterprogrammtechnik, Sprung mit Rückkehrabsicht `025`; die Schreibwerk-Ausgabe `ABC` geht nach `SW-Ausgabe/sw-ausgabe.txt`):

```
Trommeldatei:
#DOK-0001  Unterprogrammtechnik,  Sprung mit Rückkehrabsicht, 025
--sw-ausgabe SW-Ausgabe
#SPEI-0001 2012-2027
#PROT STUFE=2

0000: 00002000000 # 000 beginn ;
2000: 00004500030 # 030 04500 ; Kanal K2 anwählen
...
2022: 00000000777 # nix: dw 0777

Tastenfolge:
H G R
```

2c. Die Trommel als persistente Datei (Ergänzung 07.10.2026)

Begriffe. „Trommeldatei“ steht im Projekt für zwei verschiedene Dinge, die nicht zu verwechseln sind:

- der **Abschnitt** `Trommeldatei:` **einer Testdatei** (Abschnitt 2): die `Adresse: Wert`-Zeilen und Testmarken des Tests. Er heißt hier auch „Trommelabschnitt“;
- die **Trommeldatei** `trommel.txt` des Python-Interpreters (Vorgabe `Trommel/trommel.txt`, Schalter `--trommel`): der ganze Speicher mit 4096 Zeilen.

Das Schlüsselwort `Trommeldatei:` in der Testdatei bleibt unverändert.

Regeln für den Python-Interpreter (Einzelheiten, Meldungen und Schalter: `C8205-Interpreter-Python_Interpreter.md`, Abschnitt 8.5):

1. Die Trommel ist **dauerhaft**. Sie wird nicht je Test neu aufgebaut, sondern behält ihren Inhalt über Tests und Aufrufe hinweg. Alle Speicherzugriffe der Befehle (Lesen und Schreiben) laufen über sie; jede geschriebene Zelle steht danach auch in der Datei.
2. **Zeilenformat der Datei:** genau 4096 Zeilen `aaaa: tttttttttt`, Adressen lückenlos aufsteigend `0000` bis `7777`, keine Kommentar- und Leerzeilen (anders als im Trommelabschnitt der Testdatei).
3. **Übernahme der Testzeilen:** Vor dem Start des Tests schreibt der Interpreter die `Adresse: Wert`-Zeilen des Trommelabschnitts auf die Trommel (Überlagerung). Eine Zelle, die der Test nennt, hat danach genau diesen Wert; eine Zelle, die er nicht nennt, behält den Wert, den sie vorher hatte. Register und Flags starten dagegen bei jedem Test neu (Abschnitt 3).
4. **Formale Prüfung:** Die Trommeldatei wird einmal je Aufruf vor dem ersten Test geprüft (Abschnitt 5). Ist sie fehlerhaft oder nicht vorhanden, startet kein Test des Aufrufs.
5. **Zurücksetzen:** Ohne Testmarke setzt der Interpreter die Trommel nie zurück und legt die Datei nie an. Eine leere Trommel (alle Zellen `00000000000`) erzeugt das Hilfsskript `erzeuge_leere_trommel.py`; es überschreibt eine vorhandene Datei. Ein Test kann das Zurücksetzen selbst verlangen: `#TROMMEL=LEER` (Abschnitt 2g); nur dann, und nur im ersten Test eines Aufrufs, legt der Interpreter eine fehlende oder fehlerhafte Trommeldatei neu an.

Folgen für die Testdefinition.

- Das Ergebnis eines Tests hängt vom Trommelstand vor dem Test ab, sobald der Test eine Zelle **liest oder prüft** (Befehle, `#SPEI`/`#SPUR`, Speicherprüfzeilen in `Prüfungen:`), die er nicht selbst im Trommelabschnitt belegt. Solche Tests sind nur auf leerer Trommel reproduzierbar.
- **Empfehlung:** Ein Test nennt alle Zellen, auf die sich sein Ergebnis stützt, auch die mit Wert 0 (`aaaa: 00000000000`). Dann ist er von der Vorgeschichte unabhängig. Wo das nicht gewollt ist, steht im Test `#TROMMEL=LEER` (Abschnitt 2g), oder es wird vor dem Lauf eine leere Trommel erzeugt.
- **Reihenfolge:** Bei mehreren Tests auf derselben Trommel (mehrere Testdateien in einem Aufruf, mehrere Aufrufe nacheinander, Sammelläufe wie `testscripte/compute_all_tests.sh`) bestimmt die Reihenfolge den Anfangszustand jedes Tests. Beispiel: `bsp_seite_42_025-sw-ausgabe` prüft die Zellen `2024` bis `2027` auf 0, ohne sie zu belegen; es ist auf leerer Trommel `Test wahr`, nach `bsp_seite_40` (belegt `2024` bis `2026`) `Test falsch`.
- **Sollwerte** (Abschnitt 3a und Regressionsläufe) gelten für einen Lauf auf leerer Trommel oder für einen Test, der seine Zellen selbst nennt.
- **Protokoll:** Der Abschnitt „Ausgangsbedingungen“ (8.1) zeigt nur die vom Test geschriebenen Zeilen, nicht den ganzen Trommelinhalt vor dem Test.

Claude-Interpreter. Er hat keine Trommeldatei. Er legt nur die Zeilen des Trommelabschnitts an; Zellen, die der Test nicht nennt, gelten als 0, entsprechend einem Python-Lauf auf leerer Trommel; `#TROMMEL=LEER` ist dort ohne Wirkung (2g). Für die Gegenprobe (Abschnitt 1a) wird der Python-Lauf deshalb auf einer frisch erzeugten, leeren Trommel gestartet.

2d. Unterbrechung des Programmlaufs durch `#INTERRUPT` (Ergänzung 08.10.2026)

Ein Test kann den Ablauf der Befehlsschleife an bestimmten Stellen unterbrechen und dabei eine **weitere Tastenfolge** bedienen. Das bildet Eingriffe des Bedieners mitten im Programm nach (Halt, Register ändern, fortsetzen). Wie `#DOK`, `#SPEI`, `#PROT`, `#AC=` und `#BZ=` ist `#INTERRUPT` eine zusätzliche Definition nur für den Interpreter ohne Entsprechung in der Hardware. Die Knotentabelle bleibt unverändert.

```
#INTERRUPT  H <weitere Tasten>
```

Schreibweise. Das Argument ist eine Folge von Tasten-Token nach Abschnitt "Tastenfolge-Token" in `C8205-Interpreter-Bedienpult_mit_Schaltern.md`, durch Leerzeichen getrennt. Eine gehaltene Taste wird mit `+`, ihr Loslassen mit `-` gekennzeichnet, z. B. `#INTERRUPT H G M+ E M- R`. **Die erste Taste muss `H` sein**; danach folgen beliebige weitere zulässige Tasten. Ein Kommentar `# …` am Zeilenende ist erlaubt.

Stellung. Die Zeile steht im Abschnitt `Trommeldatei:` und **nur ZWISCHEN zwei Befehlszeilen** (`Adresse: Wert`). Sie gehört zu der unmittelbar davor stehenden Zeile mit der Adresse `p`; vor der ersten und hinter der letzten Befehlszeile ist sie unzulässig. Sie ist selbst keine Speicherzelle und kein Sprungziel: ein Sprung auf die Zelle hinter der Zeile löst sie nicht aus.

C 8205 – Durchführung eines Tests

Quelle: C8205-Interpreter-Durchführung_eines_Tests.md · im Wortlaut der Quelle

Wann sie wirkt. Die Befehlsschleife wird **nach dem Befehl der Zeile davor** unterbrochen, und zwar vor Phase I des nächsten Befehls (Zustand nach Phase IV: BZ = nächste Befehlsadresse, AR = a, OR = ω*, AC = R). Das gilt nur, wenn der Ablauf **sequentiell** weiterläuft, also nach dem Befehl auf `p` der BZ den Wert `p+1` hat (`BZ := BZ+1`, auch bei einem nicht erfüllten bedingten Sprung). Springt der Befehl woandershin, wird die Zeile vom Ablauf **übersprungen** und bleibt ohne Wirkung. Bei jedem erneuten Erreichen (Schleife) wirkt sie von Neuem. Nicht ausgelöst wird sie nach einem Befehl, der nicht aus der Zelle `p` stammt (z. B. durch `E` oder `R` aus einem aufgestellten BR ausgeführt) und nach einem Stopp (der Lauf endet dort ohnehin).

Was die Tasten bewirken. Die Tasten wirken wie in Abschnitt 3 und 3.3 auf den Zustand an der Unterbrechungsstelle, nicht auf den Anfangszustand des Tests. Register und Flags bleiben also, wie der letzte Befehl sie hinterlassen hat, und werden nur durch die Wirkung der Tasten geändert (z. B. `G`: Flags gelöscht, BR = 0000 000, US := 1; `Ü`: AR := 7777, OR := OR ∨ 240). Ohne vorheriges `G` ist OR der Operationsteil des letzten Befehls: `H Ü E` ergibt daher z. B. nach einem Ladebefehl 211 die Operation 251, nicht 240; für den reinen Transport nach 7777 ist `H G Ü E` nötig. Ein `E` ohne aufgestellten BR beginnt bei Phase I (Phase IV des vorigen Befehls ist bereits abgelaufen), mit aufgestelltem BR wie in 3.3 bei Knoten 19.

Ende oder Fortsetzung.

- Ist die **letzte Taste** `R` **oder** `ST`, geht der Lauf weiter: bei Konnektor B mit dem von den Tasten hinterlassenen BZ, bzw. bei aufgestelltem BR wie in 3.3 (Knoten 19, nach `M` mit Sprung mit Rückkehrabsicht Knoten 54). BZ und AC gelten so, wie die Tasten sie verlassen haben, ihre ändernde Wirkung wird also beachtet. Bei `ST` als letzter Taste wird H gelöscht (ST ist nur wirksam, wenn US oder BS gesetzt ist, sonst hat es keine Wirkung, der Lauf geht dann ungeändert weiter). `G` setzt BZ nicht, stellt aber BR auf 0000 000: führt ein folgendes `R` diesen Sprung aus, beginnt der Programmlauf bei 0000 von vorn (BZ := 0000, Abschnitt 3.1); folgt `M+ E … E M-`, entfällt der Sprung.
- **Jede andere letzte Taste** beendet den Test an dieser Stelle (Rechner im Halt, H gesetzt). Das Ergebnis ist der Zustand nach der letzten Taste; ein Abschnitt `Prüfungen:` gilt für diesen Zustand. Spätere `#INTERRUPT`-Zeilen kommen dann nicht mehr zum Zug.

Mehrfach. Die Zeile darf mehrfach in einer Testdatei stehen, aber höchstens einmal je Befehlszeile (je Adresse `p`).

Formale Prüfung (Abschnitt 5). Formfehler sind: `#INTERRUPT` außerhalb von `Trommeldatei:` (z. B. in `Tastenfolge:` oder `Prüfungen:`), ohne Befehlszeile davor oder dahinter, ohne Tasten, mit einer ersten Taste außer `H`, mit unbekannter Taste, mit unzulässiger M-Folge (3.3), zweimal hinter derselben Befehlszeile. Der Test startet dann nicht.

Protokoll. Im Abschnitt `== 1. Ausgangsbedingungen ==` steht je Zeile `#INTERRUPT nach Befehl pppp: Tasten`; im Ablauf (Abschnitt 2 des Protokolls) eine Zeile `--- INTERRUPT nach Befehl pppp ...` mit BZ und AC, die Wirkung jeder Taste und `--- Ende INTERRUPT ...` bei Fortsetzung.

Beispiel (nach dem Laden wird unterbrochen, der AC nach 7777 transportiert, danach endet der Test):

```
Trommeldatei:
0000: 00003000211 # 211 Laden
#INTERRUPT H G Ü E # Transport AC nach 7777, Test endet (letzte Taste E)
0001: 00003001210 # 210 Addieren
0002: 00000000020 # 020 Stopp
3000: 00000000100
3001: 00000000045

Tastenfolge:
H G R

Prüfungen:
7777: 00000000100
```

Testdateien: `int-001` bis `int-006` in `Tests/` (Fortsetzung mit `H R`, Ende ohne `R`, `G Ü E`, `G ST`, Sprung übersprungen, mehrere Zeilen).

Beispiel für Formfehler (bewusst nicht in `Tests/` abgelegt, weil der Test nicht startet und `compute_all_tests.sh` den Rückkehrcode 16 als Fehlschlag melden würde; zum Ausprobieren in eine eigene Datei kopieren):

```
Trommeldatei:
#INTERRUPT H R          # Fehler: vor der ersten Befehlszeile
0000: 00000000020
0001: 00000000020
#INTERRUPT M+ E         # Fehler: erste Taste nicht H, M bleibt gedrückt
#INTERRUPT X R          # Fehler: erste Taste nicht H, unbekannte Taste X
#INTERRUPT R            # Fehler: erste Taste nicht H
0002: 00000000020
#INTERRUPT H R          # Fehler: keine Befehlszeile dahinter
Tastenfolge:
H G R
```

Der Interpreter meldet "FORMALE PRUEFUNG NEGATIV - Test startet nicht" mit einer Zeile je Fehler und endet mit Rückkehrcode 16.

2e. Das Makro `#STARTE` für den Ansprung eines Speicherplatzes (Ergänzung 09.10.2026)

`#STARTE <Oktalzahl>` ist eine Kurzschreibweise für die Tastenfolge zum **Ansprung eines beliebigen Speicherplatzes n** (Originaldokumentation, Bedienpult, Abschnitt 4.4, Punkt 2; Beispiel B in Abschnitt 3.2):

```
#STARTE n      ≙      H G M+ E E … E M- R        (E genau n-mal)
```

Der Schritt a) der Originalvorlage nennt den Schalter H-R "in Stellung R"; das ist wie in 3.2 ein Tippfehler und gemeint ist "H". Das Makro erzeugt deshalb `H` am Anfang.

- **Wert:** `n` ist eine Oktalzahl mit 1 bis 4 Ziffern (0 bis 7777, eine Speicheradresse) und **darf nicht leer sein**. `n` ist zugleich die Zahl der Tastenanschläge `E`: `#STARTE 12` löst 10 (dezimal) Anschläge aus und spricht den Speicherplatz `0012` an.
- **Stellung:** an beliebiger Stelle der Testdatei außer im Abschnitt `Prüfungen:`, üblicherweise bei den Testmarken im Abschnitt `Trommeldatei:`; ein Kommentar `# …` am Zeilenende ist erlaubt. Höchstens einmal je Test.
- **Ersatz der Tastenfolge:** `#STARTE` ersetzt die Tastenfolge vollständig. Der Abschnitt `Tastenfolge:` darf dann fehlen (oder leer sein), enthält er Tasten, ist das ein Formfehler.
- **Wirkung:** genau die der ausgeschriebenen Tastenfolge (3.3): `G` stellt BR auf 0000 000, `M+` verknüpft den OR mit 025, das erste `E` lässt den Sprung mit Rückkehrabsicht nach 0000 laufen, BZ := 1, jedes weitere `E` zählt BZ um 1 weiter, nach dem n-ten `E` gilt `BZ = n`, `AR = n` und im AC steht im Adressfeld `n+1` (Beispiel B: n=4, AC = `0000 0005 000`). `R` setzt den Ablauf bei Konnektor B mit BZ = n fort, der erste Befehl wird aus der Zelle `n` entnommen.
- **Grenzfall `#STARTE 0`:** kein `E`, die Folge lautet `H G M+ M- R`, also ein Antippen von `M` ohne `E`. Das ist kein bloßer Start bei 0000, sondern ein aufgestellter Sprung mit Rückkehrabsicht nach 0000 (3.3); für einen schlichten Start bei 0000 dient `H G R`.
- **Protokoll:** Im Abschnitt `== 1. Ausgangsbedingungen ==` steht `#STARTE nnnn` statt der ausgeschriebenen Tasten, danach die Wirkung der erzeugten Tasten in **kompakter Form** (Ergänzung 08.10.2026): Folgen von mindestens vier `E` unter gehaltenem M erscheinen als die ersten zwei `E`-Zeilen, eine Sammelzeile `... k weitere E (BZ a bis b): je Anschlag BZ := BZ+1, AR := BZ, AC := BZ+1 im Adressfeld` und die letzte `E`-Zeile (k = Zahl der ausgelassenen Anschläge). Bei `#STARTE 1000` sind das 4 statt 512 Zeilen. `H`, `G`, `M`, `M-` und `R` bleiben vollständig. Gekürzt wird nur das Protokoll, nicht die Rechnung; der Endzustand ist unverändert. Eine ausgeschriebene Tastenfolge (`H G M+ E E … E M- R`) wird dagegen **nie** gekürzt; wer jede Taste sehen will, schreibt sie aus.
- **Formfehler (Abschnitt 5):** `#STARTE` ohne Wert, mit einem Wert, der keine Oktalzahl mit 1 bis 4 Ziffern ist, mehrfach, im Abschnitt `Prüfungen:` oder zusammen mit Tasten im Abschnitt `Tastenfolge:`. Der Test startet dann nicht.
- **Zusammenspiel:** Mit `#AC=`, `#BZ=` und `#INTERRUPT` ist `#STARTE` verträglich. `#BZ=` bleibt wie bei `H G …` bis zum ersten `E` unter M erhalten (3.3). Als Argument von `#INTERRUPT` ist das Makro nicht vorgesehen, dort wird die Tastenfolge ausgeschrieben.

Beispiel (gleichwertig zu `Tastenfolge: H G M+ E E E E M- R`):

```
Trommeldatei:
#STARTE 4
0004: 00003000211 # 211 Laden
0005: 00003001210 # 210 Addieren
0006: 00000000020 # 020 Stopp
3000: 00000000100
3001: 00000000045

Prüfungen:
BZ=0007 AC=00000000145 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
```

Testdateien: `st-001` (`#STARTE 4`), `st-002` (Gegenprobe mit ausgeschriebener Tastenfolge, gleiches Ergebnis), `st-003` (`#STARTE 12`, oktal).

2f. Tests der BIS-Bausteine: das Minimalprogramm läuft zuerst (Ergänzung 08.10.2026)

Ein Test, der einen Baustein des BIS prüft (z. B. ein Unterprogramm des Minimalprogramms wie Befehlsdruck, Adressendruck oder Triadendruck), braucht das **Minimalprogramm auf der Trommel**. Das ist für alle BIS-Tests der Normalfall und gilt als feste Regel:

1. **Schritt 1: Minimalprogramm laden.** `./c8205_interpreter.sh Tests/pj2-bis_minimalprogramm_v6.test --LOADONLY` bringt das Minimalprogramm v6 nur auf die Trommel (Sprung auf das BIS-Anfang 0040 in Zelle 0000, Sprung zum Druck von BZ und AC ab 0500 in Zelle 0001, die Unterprogramme in 0040 bis 0377). Ohne `--LOADONLY` startete die Tastenfolge H G R des Tests das Programm, und es wartete auf Tastatureingabe. Der Test ist fertig und beliebig oft wiederholbar. Er setzt vorher nichts zurück; wer eine saubere Ausgangslage braucht, erzeugt zuerst eine leere Trommel (`python3 testscripte/erzeuge_leere_trommel.py`, Abschnitt 2c).
2. **Schritt 2: der eigentliche Test.** Er liegt mit Hauptprogramm üblicherweise ab Zelle 1000, ruft das Unterprogramm über seine feste Ansprungzelle auf und endet mit einem Stopp. Er belegt **keine** Zelle des Minimalprogramms (0000 bis 0337), auch nicht 0001 (dazu die Fehlerfalle in Abschnitt 2c: Eine Testzeile `0001: …` überlagert die persistente Trommel und bleibt für spätere Tests).
3. **Dokumentation im Test:** Der Test nennt die Voraussetzung in einer `#DOK`-Zeile.
4. **Wrapper:** `testscripte/c8205_bis_test.sh TESTDATEI` führt Schritt 1 (mit `--LOADONLY`) und Schritt 2 hintereinander aus (Projektstruktur, Abschnitt 1).

Stand 24.10.2026 (Minimalprogramm v6). Die Tests des Minimalprogramms selbst, `Tests/pj2-bis_v6_*.test` (35 Stück), tragen das Abbild von v6 eingebettet und setzen `#TROMMEL=LEER`; sie laufen ohne Schritt 1 in `compute_all_tests.sh`. `c8205_bis_test.sh` dient den Tests künftiger Bausteine. Die Skripte `test.sh`, `test_bis.sh`, `test2_bis.sh`, `test3_bis.sh`, das frühere Minimalprogramm `pj2-bis_minimalprogramm.*` und die Einzeltests `pj2-bis_*.txt` samt `st-004` sind entfallen. Die folgenden Beispiele und Messungen vom 08. bis 11.10.2026 beschreiben diese früheren Fassungen und bleiben als Geschichte stehen; ihre Tests heißen jetzt `pj2-bis_v6_*`.

Aufruf eines Unterprogramms aus dem Hauptprogramm: Der Wert steht in R1 (Register 7773, `211 wert` und `240 r1`), der Aufrufer wählt den Ausgabekanal an (`030 4500` für K2, die Ausgabeunterprogramme enthalten keinen Anwahlbefehl), dann folgt `025 ansprungzelle`; die Rückkehr erfolgt in die Zelle hinter dem Aufruf (BIS-2.2.10).

> **Historisch (Stand 08. bis 11.10.2026):** Die folgenden Beispiele `pj2-bis_befehlsdruck.txt`, `pj2-bis_adressendruck.txt` und die Messung mit `pj2-bis_minimalprogramm.test` gehören zu den früheren Fassungen; diese Dateien gibt es nicht mehr. Heutige Entsprechungen: `Tests/pj2-bis_v6_druck_befehlsdruck.test`, `Tests/pj2-bis_v6_druck_adressdruck.test` und `Tests/pj2-bis_minimalprogramm_v6.test`.

Beispiel `pj2-bis_befehlsdruck.txt` (Soll-Datei `pj2-bis_befehlsdruck_erwartet.txt`): Das Hauptprogramm holt den Befehl aus Zelle 1000 (`211 1000`, Wort `00001000211`) in den AC, speichert ihn nach R1 und ruft Befehlsdruck (`025 053`). Erwartet wird die Schreibwerk-Ausgabe `1000 211` (ohne Zeilenende), R1 bleibt unverändert, Stopp in 1004 mit `BZ=1005`. V1=V2=1 stammen aus dem letzten Vergleich der Ziffernschleife (Befehl 311 mit z=0), nicht aus dem Hauptprogramm. Geprüft am 08.10.2026 auf leerer Trommel mit `c8205_bis_test.sh`: `Test wahr`, alle Einzelprüfungen und die Schreibwerk-Prüfung OK, Zelle 0001 bleibt `00000150000`. Die Schreibwerk-Ausgabe wird über `--sw-ausgabe` und `--pruefe-sw-ausgabe` in der Testdatei geprüft (Abschnitt 3b).

Beispiel `pj2-bis_adressendruck.txt` (Soll-Datei `pj2-bis_adressendruck_erwartet.txt`, Ansprungzelle 0100): Das Hauptprogramm holt das Testwort `01234567012` (`0123 4567 012`) aus Zelle 1010 in den AC, speichert es nach R1 und ruft Adressendruck (`025 0100`). Gedruckt werden nur die Wortstellen z12 bis z23, also die mittleren vier Oktalziffern;

erwartet wird die Schreibwerk-Ausgabe `4567`. R1 bleibt unverändert, weil das Unterprogramm es vor der Drehung in der Arbeitszelle 7744 sichert und am Ende zurückholt (die 12 + 12 Bit Drehung beim Druck von vier Ziffern ergeben nicht eine ganze Umdrehung von 33 Bit; beim Befehlsdruck schließen die 9 Bit des Operationsteils den Kreis). Der AC enthält am Ende das zurückgeholte R1 (der AC nach einer Ausgabe ist belanglos). Der Code des Adressendrucks liegt auf Spur 0240 (Zellen 0240 bis 0255), die Ansprungzelle 0100 enthält den Sprung dorthin. Geprüft am 08.10.2026 auf leerer Trommel mit `c8205_bis_test.sh`: `Test wahr`; der Test `pj2-bis_befehlsdruck` bleibt mit dem erweiterten Minimalprogramm `Test wahr`.

Beispiele `pj2-bis_adresseneingabe_*.txt` (Ansprungzelle 0316; je Test eine Lochstreifendatei `Test-virtuelleGeraete/Leser1/pj2-bis_adresseneingabe_<Name>.txt` und, wo gedruckt wird, eine Soll-Datei `..._erwartet.txt`): Das Hauptprogramm ab 1000 wählt Kanal K1 an (`030 4600`), ruft die Adresseneingabe (`025 0316`), speichert den AC nach R1 und druckt ihn mit dem Adressendruck (`030 4500`, `025 0100`). Die Soll-Dateien der BIS-Tests liegen in `Test-Ergebnisvergleiche/` und werden in der Testdatei mit diesem Pfad genannt (`--pruefe-sw-ausgabe Test-Ergebnisvergleiche/<Name>_erwartet.txt`, Pfad relativ zum Arbeitsverzeichnis). Die Lochstreifendateien enthalten die Zeichen im R 300-Code, eine Zeile je Zeichen im Format `w v u p 8 4 2 1` (6.2.4); Lochbilder: 0 = `00010000`, 1 = `00000001`, 2 = `00000010`, 3 = `00010011`, 4 = `00000100`, 5 = `00010101`, 6 = `00010110`, 7 = `00000111`, Punkt = `00111011`, Zwischenraum (ZR) = `00011010`.

Test	Eingabe	Erwartet
`_4stellig`	`1234.`	R1 = `00001234000`, Schreibwerk `1234`, Stopp 1005, BZ=1006
`_8stellig`	`12345670.`	R1 = `12345670000` (Wortstellen z0 bis z23 belegt), Schreibwerk `5670` (die mittleren vier Ziffern), BZ=1006
`_null`	`.`	R1 = 0 (die Konstante Null braucht nur das Schlusszeichen), Schreibwerk `0000`, BZ=1006
`_zr`	`1` ZR `0 0 5 .`	ZR bricht die Adresseneingabe ab und springt nach `kz_neu` (Zelle `ae_zr`, 0412). Die neu gestartete Kennzeicheneingabe liest $K_1 = 0$, $K_2 = 0$ und das Wort 5 und kehrt über `kz_rs` (7700, im Test mit `000 1010` vorbesetzt) nach 1010 zurück. Die Zellen 1002 bis 1005 des Hauptprogramms (Speichern nach R_1, Druck, Stopp) werden nicht erreicht. Stopp, BZ=1011, AC = `00000000005`, `kz_k1` (7701) = `kz_k2` (7702) = 0
`_ueberzaehlig`	`1234567091.`	Ab der neunten Ziffer wird alles ignoriert (auch die 9): R1 = `12345670000`, Schreibwerk `5670`, BZ=1006
`_ziffer8_halt`	`128.`	Die 8 ist keine Oktalziffer: HALT im Befehl `ae_halt` (Zelle 0257), BZ=0260, AC = `10000012000` (gelesene Konstante plus Haltmarke 2 hoch 30)
`_ziffer9_erste`	`9`	Auch als erstes Zeichen HALT: BZ=0260, AC = `10000000000`, V1=V2=1 (Gleichheit im Vergleich 330)

Die Adresseneingabe liest mit dem Eingabe-Halt `010 ae_tab` (Tabelle ab 0400, nach BIS-2.2.12.3 gehören die Plätze bis 437 zum Minimalprogramm): Jede Ziffer addiert d·0100 in den AC, danach drehen drei Sprünge `006` den AC um eine Oktalstelle nach links. Die Konstante steht damit rechtsbündig in den Oktalstellen t3 bis t10 (`0000 aaaa 000` bei vier Stellen). **Ziffern 8 und 9** sind keine Oktalziffern und führen zum **HALT**: Ihre Tabellenzellen enthalten 2 hoch 30 (`ae_pois`), der Vergleich `330 ae_pois` mit dem Sprung `150 ae_halt` erkennt das (kein gültiger AC ist an dieser Stelle so groß), und der Stopp `ae_halt` (Zelle 0257) hält an. **Jede Ziffer ab der neunten wird ignoriert**, bis das Schlusszeichen kommt, auch 8 und 9: Ein Zähler (Arbeitszelle 7760, sieben Einsen, je Ziffer um 1 Bit nach links geschoben, wie im Triadendruck) zählt acht Ziffern, danach liest die Schleife `ae_ign` jedes Zeichen mit Löschen (`011`) und verwirft es. Die Konstante wird dazu in der Arbeitszelle 7757 gehalten, der Punkt holt sie zurück (Tabellenzellen 0433 `211 ae_sv` und 0434 `000 ae_rs`); der Rückkehrbefehl liegt in 7756. Die Arbeitszellen der Adresseneingabe liegen damit auf 7756 bis 7760; die Zellen 7554 bis 7563 sind den Ansprungzellen der anderen Bausteine vorbehalten und werden vom Minimalprogramm nicht benutzt. Geprüft am 08.10.2026 auf leerer Trommel mit `c8205_bis_test.sh`: alle sieben Tests `Test wahr`, ebenso `pj2-bis_befehlsdruck` und `pj2-bis_adressendruck`.

Verlegung der Arbeitszellen (11.10.2026). Die Arbeitszellen der Ausgabe- und Eingabeunterprogramme (`okt_z`, `okt_rs`, `bd_rs`, `ad_rs`, `ad_sv`, `ae_rs`, `ae_sv`, `ae_z`) lagen auf 7554 bis 7563, den Speicherplätzen der Ansprungzellen der anderen Bausteine. Sie liegen jetzt auf 7740 bis 7744 und 7756 bis 7760 (Spezifikation `C8205-Interpreter-BIS.md`, BIS-A.3). Geprüft auf leerer Trommel mit dem neu assemblierten Minimalprogramm (`Assembler/Projekt_2/pj2-bis_minimalprogramm.test`, Kopie in `Tests/`): alle zehn Tests `pj2-bis_*` (die oben genannten sowie `_zr` und `pj2-bis_druck_bz_ac`) `Test wahr`; die Zellen 7554 bis 7563 bleiben nach allen Läufen null.

2g. Die Testmarke `#TROMMEL=LEER`, `#TROMMEL=KEEP` und `#TROMMEL=MP` (Ergänzung 13.10.2026, `KEEP` 24.10.2026)

Die Trommel ist dauerhaft (2c): Ohne weitere Angabe erbt ein Test den Trommelstand der vorigen Läufe. Mit der Testmarke `#TROMMEL=` verlangt der Test seinen Anfangszustand selbst. Er ist dann von der Vorgeschichte und von der Reihenfolge der Läufe unabhängig, auch im Einzellauf und in Sammelläufen.

Schreibweise (im Abschnitt `Trommeldatei:`, wie `#AC=` und `#BZ=`; ein Kommentar mit `#` dahinter ist zulässig):

```
#TROMMEL=LEER
#TROMMEL=KEEP
#TROMMEL=MP      # Minimalprogramm
```

Wert	Wirkung vor dem Test
`LEER`	Alle 4096 Zellen der Trommel werden 0.
`KEEP`	Die Trommel bleibt unverändert (Stand des vorigen Laufs), wie ohne Marke, aber ausdrücklich: Der Test sagt damit, dass er vom Trommelstand des Vorläufers abhängen **soll** (z. B. Treiber, die ein vorher geladenes Programm starten). Bildschirm: `Trommel: unverändert, Stand des vorigen Laufs (#TROMMEL=KEEP)`; das Protokoll nennt die Marke. Keine Wirkung auf Prüfungen, kein Überschreiben, keine Vorlagenprüfung.
`MP`	Die Trommel wird mit dem Abbild des Minimalprogramms belegt (Trommelvorlage `Trommel/Trommelvorlagen/MP.txt`, Format wie die Trommeldatei, 2c).

Standard ist `KEEP`: Fehlt die Marke `#TROMMEL=` ganz, gilt `KEEP`, die Trommel bleibt unverändert (2c); alle bisherigen Tests laufen wie vorher. `#TROMMEL=KEEP` verhält sich genauso und macht es nur sichtbar. Seit 24.10.2026 tragen die meisten vorhandenen Tests ohne `LEER`/`MP` die Zeile `#TROMMEL=KEEP` unter `#PROT STUFE=2` (Stand dieser Fassung: 242 Dateien in `Tests/`); die gleichnamigen Assembler-Quelltexte führen sie als `; #TROMMEL=KEEP` mit (`bsp_5_18_kep.asm` bleibt bei `LEER`, wie seine `.test`). Einige Tests haben noch keine Marke (26 `.test`-Dateien, darunter `bsp_5_15`, `bsp_5_16`, `bedsprung`, `code`, `mini`, `sortierung`, `ziffern`, die `deepseek_asm_*`, die `pj2_*` und die `pj2-bis_minimalprogramm_v6_test_*`); für sie gilt die Standardregel, also `KEEP`. Für neue Tests gilt: entweder `LEER` (unabhängig von der Vorgeschichte) oder `KEEP` (abhängig vom Vorläufer, gewollt); eine fehlende Marke bedeutet dasselbe wie `KEEP`, sollte aber nicht mehr vorkommen.

Ablauf (Python-Interpreter):

1. formale Prüfung des Tests (Abschnitt 5);
2. die Trommel wird neu gesetzt; die Trommeldatei wird dabei vollständig neu geschrieben;
3. die Zeilen des Abschnitts `Trommeldatei:` überlagern danach den Inhalt (2c, Regel 3);
4. Tastenfolge und Programmlauf wie üblich.

Die Reihenfolge 2 vor 3 ist zwingend: Zellen, die der Test selbst nennt, haben ihren Wert; alle übrigen sind 0 (`LEER`) bzw. haben den Wert des Abbilds (`MP`). Register, Flags und Geräte sind davon nicht berührt; sie beginnen ohnehin bei jedem Test neu.

`MP`. Das Abbild des Minimalprogramms ist eine Datei im Format der Trommeldatei (2c): `Trommel/Trommelvorlagen/MP.txt` (mit `MP.basp`). Es entsteht aus dem fertigen Minimalprogramm; im Projekt ist das der Trommelstand nach dem Laden mit `Tests/pj2-bis_minimalprogramm_v6.test` (`--LOADONLY`) auf leerer Trommel (2f). Mit `#TROMMEL=MP` entfällt der Vorlauf mit `c8205_bis_test.sh`; Beispiel: `interpreter_test.sh` (2k). Fehlt die Vorlage, ist `#TROMMEL=MP` formal ungültig: Der Test startet nicht (Rückkehrcode 16) und meldet `#TROMMEL=MP: Trommelvorlage (Minimalprogramm) nicht gefunden: …`. Der Interpreter erzeugt die Vorlage nie selbst.

Fehlende oder fehlerhafte Trommeldatei. Steht `#TROMMEL=LEER` (oder `#TROMMEL=MP` mit gültiger Vorlage) im **ersten** Test eines Aufrufs und ist die Trommeldatei nicht vorhanden oder formal fehlerhaft (2c, Regel 4), so ist ihr alter Inhalt ohne Belang: Der Interpreter legt sie neu an (4096 Zeilen, alle Zellen 0; fehlende Ordner werden angelegt) und meldet das in einer Hinweiszeile, statt mit Rückkehrcode 16 zu enden. Steht die Marke erst in einem späteren Test, gilt die Prüfung vor dem ersten Test wie bisher.

Der bisherige Inhalt geht verloren. Der Test überschreibt die verwendete Trommeldatei ohne Rückfrage. Deshalb zeigt der Bildschirm die Zeile `Trommel: vor dem Test geleert (#TROMMEL=LEER)` (bei `MP`: `… mit dem Abbild "Minimalprogramm" belegt (#TROMMEL=MP)`), und das Protokoll (8.1, Ausgangsbedingungen) nennt die Marke. Liegt auf der Standard-Trommel ein Arbeitsstand, den man behalten will, bekommen solche Tests mit `--trommel PFAD` eine eigene Trommeldatei.

Grenzen. Die Marke stellt nur den leeren Zustand oder ein Abbild her. Braucht ein Test einen anderen Trommelinhalt, so nennt er die Zellen selbst im Abschnitt `Trommeldatei:` oder stellt den Inhalt durch einen

vorherigen Lauf her. So lädt `compute_all_tests.sh` vor `st-005` die Druckroutine aus `asm_druck_tastenfolge.txt`; die Druckroutine ist ein bewusst vereinfachter Sonderfall und wird von Fall zu Fall hergerichtet.

Claude-Interpreter. Er hat keine Trommeldatei; für nicht genannte Zellen gilt 0 (2c). `#TROMMEL=LEER` hat dort daher keine Wirkung. `#TROMMEL=MP` gilt entsprechend, sofern ihm das Abbild vorliegt; sonst startet der Test nicht (Grundsatz „laut scheitern“, Abschnitt 5).

Beispiel: `Tests/trommel-001_leer.txt` belegt zwei Zellen selbst und prüft, dass drei weitere Zellen 0 sind. Er ist auch auf einer verschmutzten Trommel `Test wahr`.

2h. Beispiel 1: Ein Programm über Lochstreifen auf die Trommel laden und in TEST und GRAFIK ausführen (Ergänzung 20.10.2026)

Dieser Abschnitt führt einen Test durch, dessen Programm **nicht** in den Zeilen der Trommeldatei steht, sondern von einem Lochstreifen in die Trommel geladen wird. Er verbindet die Testdatei (Abschnitt 2 und 2b), die Hilfsprogramme für Lochstreifen (`C8205-Interpreter-Lochstreifen-Hilfsprogramme.md`), den TEST-Interpreter und den GRAFIK-Interpreter. Als Beispiel dient `Tests/k1_leser_stanzer.test`: Das Programm liest von Leser 1 Zeichen bis zum Zeichen `:` und stanzt sie auf Stanzer 1 (Lochstreifen kopieren). Im Beispiel enthält der Streifen den Text `Hallo Welt 123 (ok)!:`.

Warum dieser Umweg. Ein Programm gelangt bisher nur über den Test auf die Trommel; das Minimalprogramm v6 lädt die Objektkode-Lochstreifendateien des Assemblers zwar direkt (Weg 4 in 2k), dieses Beispiel nimmt aber den Weg über die Testdatei. Der Lauf mit dem TEST-Interpreter lädt das Programm nach den Zeilen des Abschnitts `Trommeldatei:` auf die Trommel (2c) und prüft zugleich das Ergebnis. Die Trommel (`Trommel/trommel.txt`) benutzen TEST- und GRAFIK-Interpreter gemeinsam; das Programm steht danach auch für GRAFIK bereit.

Die Geräte. Der TEST-Interpreter benutzt die Geräteordner unter `Test-virtuelleGeraete/`, der GRAFIK-Interpreter die unter `virtuelle_Geraete/`. Die beiden Ordner sind getrennt, ein Lochstreifen muss also in beide gelegt werden (Schritt 6).

2h.1 Schritt für Schritt

1. Testdatei definieren und nach `Tests/` legen. Die Geräte-Zeilen (2b) nennen den Lochstreifen, den der Leser liefert, und die Datei, in die der Stanzer schreibt:

```
Trommeldatei:
#DOK-0001
#SPEI-0001 0000-0000
#PROT STUFE=2
--leser1 Test-virtuelleGeraete/Leser1_02/lb_kopie_quelle.txt
--stanzer1 Test-virtuelleGeraete/Stanzer1/lb_kopie_ausgabe.txt
0000: 00002000000 # beginn: 000 anfang ; Zelle 0000: Sprung nach 02000
2000: 00004600030 # anfang: 030 04600 ; 4600 oder K1 oder LSL1 oder LSS1
...
```

Der Stanzer-Ausgabe wird in der Testdatei eine Soll-Datei zugeordnet (`Test-Ergebnisvergleiche/k1_leser_stanzer-stanzer-ausgabe.txt`, Abschnitt 3b); der Test prüft am Ende, ob der Stanzer dieselben Zeichen ausgegeben hat.

2. Den Lochstreifen erzeugen, den der Leser im Test liefern soll. Er muss als binär codierter R300-Lochstreifen in der Datei liegen, die die Zeile `--leser1` nennt: `Test-virtuelleGeraete/Leser1_02/lb_kopie_quelle.txt`. Format: je Lochungszeile 8 Ziffern 0/1 (Hilfsprogramme, Abschnitt 2). Für `Hallo Welt 123 (ok)!:` sind das 29 Zeilen, beginnend mit

```
00101111
00111000
01001111
00110001
...
```

Es gibt drei Wege; sie führen zu derselben Datei.

Weg A: Klartext, dann Umwandlung (empfohlen). Der Text steht in einer gewöhnlichen Textdatei, z. B. `lb_kopie_quelle.text` (nur die Zeile `Hallo Welt 123 (ok)!:`). Die Umwandlung stanzt daraus den Lochstreifen; `--ls-leser` nennt die Textdatei, `--ls-stanzer` den zu erzeugenden Lochstreifen:

```
$ python3 Hilfsprogramme/c8205_lochstreifen_umwandlung.py --ls-leser lb_kopie_quelle.text \
      --ls-stanzer Test-virtuelleGeraete/Leser1_02/lb_kopie_quelle.txt --in-bin
```

Weg B: Eingabefenster. Der Text wird im Fenster des Eingabe-Hilfsprogramms getippt; jedes Zeichen wird sofort als Lochungszeile in die genannte Datei geschrieben. Das Programm erzeugt also **sofort den binären Lochstreifen**; eine Umwandlung entfällt:

```
$ python3 Hilfsprogramme/c8205_lochstreifen_eingabe.py \
      --ls-stanzer Test-virtuelleGeraete/Leser1_02/lb_kopie_quelle.txt
```

Beenden mit **X**; es gibt kein Speichern, die Datei ist immer auf dem Stand des Fensters (Hilfsprogramme, Abschnitt 4.2).

Weg C: von Hand. Die Datei mit den Lochungszeilen (8 Ziffern je Zeile) in einem Texteditor schreiben und nach `Test-virtuelleGeraete/Leser1_02/` legen.

3. Lochstreifen kontrollieren. Die Anzeige schreibt den Streifen als Klartext aus:

```
$ python3 Hilfsprogramme/c8205_lochstreifen_anzeige.py \
      --ls-leser Test-virtuelleGeraete/Leser1_02/lb_kopie_quelle.txt --text
Hallo Welt 123 (ok)!:
```

Ohne `--text` erscheint die Ausgabe in einem Fenster.

4. Der Leser des TEST-Interpreters ist damit belegt. Der Ort steht in der Zeile `--leser1` der Testdatei; es ist nichts weiter einzulegen:

```
$ ls Test-virtuelleGeraete/Leser1_02/lb_kopie_quelle.txt
Test-virtuelleGeraete/Leser1_02/lb_kopie_quelle.txt
```

5. Den TEST-Interpreter ausführen. Der Lauf schreibt das Programm aus der Testdatei auf die Trommel, führt es aus und gibt das Ergebnis gemäß den Prüfungen der Testdatei (3a) und der Dateiprüfung des Stanzers (3b) aus:

```
$ python3 c8205_interp.py --MODUS=TEST Tests/k1_leser_stanzer.test

=== Test k1_leser_stanzer (#PROT STUFE=2) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 6137 | BZ=4035 AC=00000000020 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  BZ: soll=4035 ist=4035 OK
  AC: soll=00000000020 ist=00000000020 OK
  ...
  Stanzer 1: soll=Test-Ergebnisvergleiche/k1_leser_stanzer-stanzer-ausgabe.txt
    ist=Test-virtuelleGeraete/Stanzer1/lb_kopie_ausgabe.txt OK
Test wahr
Protokolldatei: Test-Protokolle/k1_leser_stanzer/protokoll.txt
```

Das Programm steht jetzt auf der gemeinsamen Trommel `Trommel/trommel.txt`. Der Stanzer hat den Text nach `Test-virtuelleGeraete/Stanzer1/lb_kopie_ausgabe.txt` kopiert.

6. Für den GRAFIK-Interpreter den Lochstreifen in dessen Leser legen. Der Leser der GRAFIK-Umgebung ist ein Ordner mit nachrückenden Lochstreifendateien (`virtuelle_Geraete/Leser1/`, siehe GRAFIK-Beschreibung); der Streifen aus der Testumgebung wird dorthin kopiert:

```
$ cp Test-virtuelleGeraete/Leser1_02/lb_kopie_quelle.txt virtuelle_Geraete/Leser1/
```

7. Das Programm im GRAFIK-Interpreter ausführen. Das Programm liegt auf der Trommel und beginnt bei Adresse 0000. Es kopiert von Leser nach Stanzer, deshalb werden beide Geräte angegeben:

```
$ ./c8205_grafik.sh --leser1 virtuelle_Geraete/Leser1 --stanzer1 virtuelle_Geraete/Stanzer1 --HWCHECK
```

Am Pult: den Schlüsselschalter (Netzschalter) anklicken, wenn der Rechner bereit ist; auf **R** schalten, dann wird der Lochstreifen kopiert. Zum Beenden den Schlüsselschalter wieder anklicken.

Nach dem Lauf ist die gelesene Datei `lb_kopie_quelle.txt` aus `virtuelle_Geraete/Leser1/` verschwunden und liegt in `virtuelle_Geraete/Leser1/LOCHSTREIFENKORB/`. Der Stanzer schreibt die kopierten Lochungszeilen in `virtuelle_Geraete/Stanzer1/stanzer.ls` fort (Ansicht mit der Anzeige: `--ls-leser virtuelle_Geraete/Stanzer1/stanzer.ls`).

2h.2 Hinweise

- Das Verfahren ist für jedes Programm gleich, das über Lochstreifen Daten verarbeitet: Testdatei mit `--leser1`/`--leser2`/`--stanzer1`, Streifen im Leser des TEST-Interpreters, Testlauf, danach Streifen in den Ordner des GRAFIK-Lesers kopieren.

- Der TEST-Interpreter verbraucht den Streifen nicht: Die Datei in `Test-virtuelleGeraete/Leser1_02/` bleibt erhalten und kann für weitere Läufe benutzt werden. Der GRAFIK-Interpreter verschiebt gelesene Dateien dagegen in den `LOCHSTREIFENKORB`; für einen weiteren Lauf muss man die Datei von dort zurückkopieren.

2i. Beispiel 2: Ein Assemblerprogramm anlegen, assemblieren und im GRAFIK-Interpreter ausführen (Ergänzung 20.10.2026)

Der zweite Arbeitsablauf geht vom Quelltext aus: Ein Assemblerprogramm wird angelegt, assembliert, mit dem TEST-Interpreter auf die Trommel geladen und im GRAFIK-Interpreter bedient. Als Beispiel dient `namen_raten`, ein Ratespiel über das Schreibwerk (SW): Das Programm wählt einen von fünf Namen mit je 7 Buchstaben, nennt den 1. Buchstaben, und der Benutzer tippt die übrigen Buchstaben am SW. Ein Punkt beendet das Programm.

Anders als in Beispiel 1 (2h) braucht dieses Programm keinen Lochstreifen. Der TEST-Interpreter dient wieder nur dazu, das Programm auf die gemeinsame Trommel (`Trommel/trommel.txt`) zu laden (Grund und Einschränkung wie in 2h); die eigentliche Ausführung geschieht im GRAFIK-Interpreter.

2i.1 Schritt für Schritt

Die Pfade gelten vom Projektordner aus (dort, wo `c8205_interp.py` liegt).

1. Assembler-Projekt anlegen. Die Quelltexte liegen unter `Assembler/`, je Projekt ein Ordner:

```
$ cd Assembler
$ mkdir -p ./Projekt_1
```

2. Unterprojekt anlegen (optional). Je Programm ein eigener Ordner im Projekt; in ihm entstehen auch die Ausgabedateien des Assemblers:

```
$ mkdir -p ./Projekt_1/namen_raten
```

3. Assembler-Quelltext erfassen mit einem Editor Ihrer Wahl, hier `code`:

```
$ code ./Projekt_1/namen_raten/namen_raten.asm
```

4. Assemblieren und die Testanforderungsdatei erzeugen. Der Aufruf geschieht im Ordner `Assembler/`. `-p` nennt das Projektverzeichnis, `-s` den Quelltext darin, `-t` den Ordner, in den die Testanforderungsdatei geschrieben wird (hier `Tests/` des Projekts); `-WARN=SYM` meldet unbenutzte Symbole:

```
$ ./c82ass.sh -WARN=SYM -p Projekt_1/namen_raten -s namen_raten.asm -t ../Tests
...
Pruefung auf unbenutzte Symbole (-WARN=SYM)
Warnung:  Symbol 'beginn' (0000, namen_raten.asm Zeile 33) ist definiert, wird aber nie verwendet.
1 unbenutztes Symbol.


Ausgabe als binaer codierte Objektdatei (R300-Lochstreifen) nach: Projekt_1/namen_raten/namen_raten.obj
Ausgabe der formatierten Speicherbelegung nach: Projekt_1/namen_raten/namen_raten.dmp
Ausgabe der Testdatei (Spalten 1,3,4,5 fuer K1=00/01) nach: ../Tests/namen_raten.test
```

Die Warnung ist hier harmlos: Die Marke `beginn` bezeichnet die Zelle 0000 und wird von keinem Befehl angesprungen. Ob die Testanforderungsdatei richtig liegt, zeigt

```
$ ls ../Tests/namen_raten.test
../Tests/namen_raten.test
```

(Zur Testanforderungsdatei aus einem Assemblerlauf siehe Abschnitt 2a.)

5. TEST-Interpreter starten, um das Programm auf die Trommel zu laden. Im Projektordner:

```
$ cd ..
$ python3 c8205_interp.py --MODUS=TEST Tests/namen_raten.test

=== Test namen_raten (#PROT STUFE=2) ===
Rate mal, welchen Namen ich mir gemerkt habe.
Mit der Eingabe eines Punktes beendest du das Programm.
1. Buchstabe ist:  S
--- Achtung: Terminaleingaben erforderlich (K0 ohne Umleitung --k0); Abbruch mit Strg+C ---
```

Der Interpreter wartet nun im Terminal, weil das Programm Eingaben über das Schreibwerk verlangt. Diese Eingaben müssen im TEST-Interpreter **nicht** nachgestellt werden: Manuelle Eingaben mit zufälligem Charakter (das Programm wählt den Namen zufällig) lassen sich in einem Test nicht sinnvoll vorgeben, und der Lauf dient hier nur

dem Laden des Programms. Die Trommel ist zu diesem Zeitpunkt bereits beschrieben. Der Lauf wird deshalb mit **Strg+C** abgebrochen.

> **Richtig laden mit** `--LOADONLY` **(Ergänzung 21.10.2026):** Wenn es nur darum geht, das Programm auf die Trommel zu bringen, genügt der Schalter `--LOADONLY`. Der TEST-Interpreter überträgt dann das Programm auf die Trommel und endet sofort mit Rückkehrcode 0, ohne das Programm zu starten; es wartet nichts auf Eingaben, und Strg+C entfällt:
> `` $ python3 c8205_interp.py --MODUS=TEST --LOADONLY Tests/namen_raten.test ``
> Das gilt für jedes Programm, nicht nur für solche, die gleich zur Eingabe übergehen. Wurde `--LOADONLY` vergessen, hilft wie oben Strg+C: Die Trommel ist bereits beschrieben. Schalter: Python-Interpreter, Abschnitt 5.1.

6. GRAFIK-Interpreter starten und das Programm anspringen. Das Programm steht auf der gemeinsamen Trommel und beginnt bei Adresse 0000. Es braucht keine Geräte-Schalter; `--HWCHECK` blendet das zusätzliche Fenster mit den Registern ein:

```
$ ./c8205_grafik.sh --HWCHECK
```

Am Pult: den Rechner mit dem Schlüsselschalter (Netzschalter) einschalten; bei gedrückter Taste **H** die Wippe **T-SW** von T auf **SW** stellen (die Eingaben kommen damit aus dem SW-Fenster) und dann **R** drücken. Das Programm läuft an. Die Eingabe der Buchstaben erfolgt im SW-Fenster; ein Beispiel für die Ausgabe:

```
Rate mal, welchen Namen ich mir gemerkt habe.
Mit der Eingabe eines Punktes beendest du das Programm.
1. Buchstabe ist:  M
   Falsch, noch ein Versuch.
   Du musst deine Fantasie viel mehr schulen.
...
```

Der erste Buchstabe wechselt von Spiel zu Spiel, weil das Programm den Namen zufällig wählt. Ein Punkt beendet das Programm.

2i.2 Unterschied zu Beispiel 1

	Beispiel 1 (2h)	Beispiel 2 (2i)
Programm	`k1_leser_stanzer`: von Hand in der Testdatei geschrieben	`namen_raten`: Assemblerquelltext, vom Assembler übersetzt
Testdatei	von Hand erstellt	vom Assembler erzeugt (`-t`, 2a)
Lochstreifen	ja: Leser und Stanzer	nein: Ein- und Ausgabe über das Schreibwerk (SW)
TEST-Interpreter	lädt das Programm und prüft das Ergebnis	lädt das Programm (am besten mit `--LOADONLY`; ohne den Schalter wird der Lauf mit Strg+C abgebrochen)
GRAFIK-Interpreter	mit `--leser1` und `--stanzer1`, Schlüsselschalter und R	ohne Geräte-Schalter, bedient wird am Pult und im SW-Fenster

2j. Beispiel 3: Ein absolut assembliertes Programm als binären Lochstreifen mit Hilfe des KEP laden (Ergänzung 22.10.2026)

Der dritte Arbeitsablauf lädt das Programm nicht über den TEST-Interpreter, sondern so, wie es am Gerät geschieht: Das Kleine Eingabeprogramm (KEP) liest einen binär codierten Lochstreifen (`.kep`) vom Leser 1 und schreibt die Worte auf die Trommel. Der TEST-Interpreter bringt nur das KEP selbst auf die Trommel (Schalter `--LOADONLY`). Als Programm dient wieder `namen_raten` (Beispiel 2, 2i). Die ausführlichen Erläuterungen stehen im BIS-Dokument (BIS-2.2.11b); hier steht nur der Ablauf.

2j.1 Schritt für Schritt

Die Aufrufe der Schritte 1 und 2 gelten vom Ordner `Assembler/` aus, die übrigen vom Projektordner (dort, wo `c8205_interp.py` liegt). Alle Pfade sind relativ zum jeweiligen Ordner.

1. Das Programm absolut assemblieren. Der Quelltext muss absolut geschrieben sein und vor `start` das Pragma `;#; LEITADRESSE 00` tragen; `start` steht ohne Operand (bei `namen_raten` schon so). Der Schalter `--ABSPR` erzeugt statt der `.obj` den Lochstreifen `namen_raten.kep`, `-o` legt ihn in den Ordner des Lesers:

```
$ cd Assembler
$ ./c82ass.sh -p Projekt_1/namen_raten -s namen_raten.asm -o ../virtuelle_Geraete/Leser1 --ABSPR
$ cd ..
```

Die Warnung zur Bahn 0000 bis 0037 ist hier Absicht: Zelle 0000 trägt den Sprung zur Startadresse. Die Meldung „kein Kennzeichen 01" ist mit dem Pragma nur ein Hinweis.

2. Das KEP auf die Trommel bringen (nur nötig, wenn es nicht schon dort steht; Assemblieren des KEP mit `-t ../Tests` siehe BIS-2.2.11b, Schritt 1):

```
$ python3 c8205_interp.py --MODUS=TEST Tests/bsp_5_18_kep.test --LOADONLY
```

Der Test wird nicht ausgeführt. Die Testdatei trägt `#TROMMEL=LEER`: Die Trommel `Trommel/trommel.txt` wird vorher geleert, danach steht nur das KEP auf ihr (Zelle 0001 und alle übrigen Zellen sind 0).

3. Den GRAFIK-Interpreter starten:

```
$ ./c8205_grafik.sh --HWCHECK
```

4. Das KEP ansprechen: `H G M+ E E M- K1 R`. Der Leser 1 wird angewählt, das KEP liest den Streifen aus `virtuelle_Geraete/Leser1/`. Nach dem Lesen stehen die Worte auf der Trommel, und das Programm ist ansprungbereit (Zelle 0000 springt zur Startadresse).

2j.2 Hinweise

- **Lochstreifenkorb.** Der gelesene Streifen wandert aus `virtuelle_Geraete/Leser1/` nach `LOCHSTREIFENKORB/`; vor einem erneuten Laden die Datei zurücklegen.
- **Erneutes Laden.** Das KEP bleibt auf der Trommel stehen, Schritt 2 entfällt dann.
- **Unterschied zu Beispiel 2 (2i).** Dort lädt der TEST-Interpreter das Programm selbst auf die Trommel; hier liest das KEP es vom Streifen, so wie am Gerät.

2k. Lademethoden und ihr Angang am Assemblerquelltext (Ergänzung 24.10.2026)

Ein Programm kommt auf vier Wegen auf die Trommel. Jeder Weg verlangt etwas anderes vom Quelltext und vom Assembleraufruf; wer den falschen Quelltext mit dem falschen Weg paart, bekommt falsche Adressen oder ein Programm, das nicht anspringt. Die Beispiele 1 bis 3 (2h bis 2j) zeigen je einen Weg im Einzelnen; dieser Abschnitt stellt alle nebeneinander. Die Einzelheiten der Schalter stehen in der Assemblerdokumentation (`.odt` maßgeblich), Abschnitt 3.1a (`--ABSPR`) und 3.1b (Testmarken im Quelltext).

2k.1 Übersicht

Weg	Was der Quelltext braucht	Assembleraufruf (von `Assembler/`)	Beispielskript
Testdatei mit `--LOADONLY` (KEP, Minimalprogramm v6)	Testmarken als Kommentarzeilen am Zeilenanfang (`; #TROMMEL=LEER` oder `; #TROMMEL=KEEP`, `; #SPEI-…`, `; #PROT STUFE=2`); der Assembler übernimmt sie in die `.test` (3.1b)	`-t ../Tests/<name>.test`	`kep_test.sh`, `bis_test.sh`
Testdatei mit `#TROMMEL=MP`	`; #TROMMEL=MP` im Quelltext; das Minimalprogramm kommt aus der Vorlage (2g)	`-t ../Tests/<name>.test`	`interpreter_test.sh`
KEP-Lochstreifen `.kep` (Leser 1, KEP liest ihn)	absolut adressiert, Pragma `;#; LEITADRESSE 00` **vor** `start` (3.3.12), `start` ohne Operand	`-o ../virtuelle_Geraete/Leser1/<name>.kep --ABSPR`	`kep_test.sh` (Druckprogramm `…-KEP.asm`)
BIS-Objektstreifen `.obj` (Leser 1, das BIS liest ihn)	**kein** Pragma `LEITADRESSE`; das Kennzeichen 01 wirkt mit der Hauptleitadresse, die man am Fenster eingibt	`-o ../virtuelle_Geraete/Leser1/` (Name aus `-s`)	`bis_test.sh` (Druckprogramm `…-BIS.asm`)

Das BIS selbst (Minimalprogramm v6) kommt entweder mit dem TEST-Interpreter und `--LOADONLY` auf die Trommel (Zeile 1) oder, so wie am Gerät, als `.kep` über das KEP und Leser 2 (Zeile 3, `--ABSPR`). Das KEP lädt der TEST-Interpreter immer selbst; das KEP ist der einzige Teil, der nie über einen Streifen kommt.

Pragma bei v6 als `.kep`. Auch `pj2-bis_minimalprogramm_v6.asm` trägt `;#; LEITADRESSE 00` und braucht es wegen seiner `equ`-Zeilen: Ohne das Pragma änderten sich 92 von 275 Einträgen der `.kep`.

2k.2 Die drei Quelltextfassungen des Druckprogramms

Das Hauptprogramm „alle Druckunterprogramme“ liegt in `Assembler/Projekt_2/` in drei Fassungen, die sich nur im Angang unterscheiden:

Datei	Weg	Besonderheit
`pj2-bis-minimalprogramm-alle-druck-up.asm`	Testdatei, `#TROMMEL=MP`	`start` mit `equ 02000`; die Tastenfolge `H G R` und der Startbefehl kommen aus der Testdatei und der K0-Datei (`10 0000 2000 000 .`)
`pj2-BIS-alle-druck-up-KEP.asm`	KEP-Streifen `.kep`	Pragma `;#; LEITADRESSE 00`, absolut, Zelle 0000 trägt den Sprung nach 2000
`pj2-BIS-alle-druck-up-BIS.asm`	BIS-Streifen `.obj`	kein Pragma, relativ zur eingegebenen Hauptleitadresse (`21 3000`)

2k.3 Der BIS-Weg Schritt für Schritt (`bis_test.sh`)

1. Trommel leeren; KEP und BIS (v6) mit `-t` assemblieren und mit `--LOADONLY` laden (die Testdateien tragen `KEEP` bzw. `LEER`, siehe 2g).
2. Das Druckprogramm als `.obj` in den Ordner von Leser 1 assemblieren, das BIS (v6) zusätzlich mit `--ABSPR` als `.kep` in den Ordner von Leser 2.
3. GRAFIK starten (`./c8205_grafik.sh --HWCHECK`). Das KEP ansprechen: `H G M+ E E M- K2 R` (Kanal vor `R`, wie in 2j.1); es liest das BIS von Leser 2.
4. Das BIS ansprechen: `H G R`. Als Leitadresse für das Druckprogramm `21 3000.` eingeben (ohne Leerzeichen), dann `K1` wählen; der Streifen wird geladen. Am Ende steht das BIS wieder in der Kennzeicheneingabe.
5. Startadresse eingeben: `K0` und `10 3000 000 .` (ohne Leerzeichen). Das Druckprogramm prüft die drei Druckunterprogramme des BIS und hält an.

Der automatisierte Test zu diesem Weg ist `Tests/pj2-bis_v6_obj_laden.test` (Programm auf 3000 bis 3021).

2k.4 Fallen am Quelltext

- **Pragma und Weg müssen zusammenpassen.** `.kep` verlangt `LEITADRESSE 00`, `.obj` für das BIS darf es nicht tragen. Beim Pragma ersetzt der Assembler das Kennzeichen 01 durch 00 (absolut); fehlt es, bleibt 01 und die Hauptleitadresse wirkt.
- **Das Pragma steht vor** `start` (3.3.12), nicht zwingend in der ersten Zeile.
- **Startbefehl als Wort.** Der Befehl steht mit der Adresse links vor dem Befehlscode: `10 2000000.` für „Sprung nach 2000“, nicht `10 2000`.
- **Testmarken** zählen nur, wenn die Zeile mit `;` beginnt (3.1b); ein Kommentar hinter einem Befehl wird nicht übernommen.
- **Nur ein Streifen im Leserordner.** `kep_test.sh` legt `…-KEP.kep`, `bis_test.sh` legt `…-BIS.obj` nach `virtuelle_Geraete/Leser1/`; beide Skripte räumen den Ordner nicht auf. Liegen nach beiden Läufen beide Streifen dort, ist nicht gesagt, welchen der Leser nimmt: den nicht benötigten Streifen vorher entfernen (der gelesene wandert ohnehin in den `LOCHSTREIFENKORB`).
- **Die `.obj` ist binär.** Sie ist ein R300-Lochstreifen (Lochbildzeilen 0/1), kein Text; sie entsteht nur mit dem Assembler ab 1.00-036.

2k.5 Die drei Skripte in `Testausfuehrungen/`

Skript	Was es tut	Prüfung
`interpreter_test.sh`	Trommel leeren, Druckprogramm mit `#TROMMEL=MP` assemblieren, mit dem TEST-Interpreter ausführen, Protokollende zeigen	„Test wahr“
`kep_test.sh`	KEP und v6 mit `--LOADONLY` laden, Druckprogramm als `.kep` nach Leser 1, danach GRAFIK; Bedienung wie 2j.1 (Ausgabe `4567`, `0765 432`, `0765 4321 065`, Halt bei BZ=2017)	am Fenster
`bis_test.sh`	KEP laden, Druckprogramm als `.obj` nach Leser 1, BIS als `.kep` nach Leser 2, GRAFIK; Bedienung wie 2k.3	am Fenster

Die Skripte wechseln selbst in den Projektstamm und laufen von überall (Aufruf `bash Testausfuehrungen/<name>.sh`).

3. Schritt 2 jedes Tests: die Tastenfolge zur Definition des Ausgangszustands

Nach Übernahme einer korrekten Trommeldatei wird eine Tastenfolge angegeben, die auf die Ebene von Registern (mit ihren Bits) und Flags (mit ihren Werten 0/1) herunterzubrechen ist. Dadurch wird der Ausgangszustand des Rechners vor Eintritt in Konnektor B vollständig definiert.

3.1 Beispiel A — Ansprung des Speicherplatzes 0000 (direkt)

Trommeldatei wie in Abschnitt 2.

Tastenfolge: `H G R`

Schritt	Aktion	Wirkung auf Register/Flags
1	Schalter H-R → "H"	Flag H := 1
2	Taste "G"	alle Flags außer H, T und SP werden gelöscht (auch das Flag BS; der Schalter BS bleibt eingerastet), danach US := 1 (s. Fall 1); BR wird auf "0000 000" gesetzt (unbedingter Sprung nach Speicherplatz 0000); K0 wird angewählt, Schalter "SW" ist nicht gedrückt → Flag T := 1; **BZ und AC bleiben unverändert** (G hat laut Dokument "keinen Einfluss auf ... Befehlszählerinhalt"). `<BZ>:=0` ist erst das Ergebnis der Ausführung des durch G aufgestellten Befehls "0000 000" (ANWEISUNG-007, `<BZ>:=a`, hier a=0000), also des Sprungs nach 0000. Dieser Sprung wird ausgeführt, wenn auf G kein `M+ E … E M-` folgt, hier durch `R` (Schritt 3). Der Interpreter setzt BZ := 0 deshalb erst bei `R` bzw. bei `E` ohne `M`, nicht bei G (Ergänzung 09.10.2026). Gibt der Test `#BZ=` vor (Abschnitt 2), bleibt dieser Wert nach G bis zur Ausführung des Sprungs erhalten
3	Schalter H-R → "R"	Flag H := 0, US := 0; der von G aufgestellte BR "0000 000" wird ausgeführt: Sprung nach 0000, `<BZ>:=0000`; Interpretation setzt bei Konnektor B fort, mit BZ=0000

Prinzip: Tastenfolgen werden nicht nur als Rohschalterzustände, sondern bis zur Wirkung der dadurch geladenen/ausgelösten Instruktion vorausinterpretiert, damit der Automat ab Konnektor B im regulären 4-Phasen-Zyklus weiterlaufen kann, ohne dass der manuelle Eingriff selbst als Sonderfall im Knotengraphen behandelt werden müsste.

3.2 Beispiel B — Ansprung eines beliebigen Speicherplatzes (hier: 0004)

Trommeldatei (abweichende Belegung):

```
# Beginn Programmbefehle
0004: 00003000211
0005: 00003001210
0002: 00000004020
# Beginn Datenbereich
3000: 00000000100
3001: 00000000045
```

Tastenfolge: `H G` `M`-gedrückt-halten `E E E E` `M`-loslassen `R`

(Entspricht Dokument "C8205-Interpreter-Bedienpult_mit_Schaltern.md", Abschnitt 4.4, Punkt 2 — "Ansprung eines beliebigen Speicherplatzes n", hier n=4. Anmerkung: Im Originaltext steht bei diesem Fall als Schritt a) "Schalter H-R → R"; das ist im Kontext, da G/M/E nur bei Stellung "H" wirksam sind, vermutlich ein Tippfehler und müsste "→ H" heißen, wie in der hier verwendeten Tastenfolge auch tatsächlich benutzt. Korrigiert am 02.10.2026: im Bedienpult-Dokument, Abschnitt 4.4, steht jetzt "→ H".)

Schritt	Aktion	Wirkung auf Register/Flags
1	H-R → "H"	Flag H := 1
2	Taste "G"	alle Flags außer H, T, SP := 0 (auch BS); BR := "0000 000"; K0 angewählt, T := 1; BZ und AC bleiben unverändert (Originaldokumentation; G setzt BZ nicht, der Sprung nach 0000 des aufgestellten BR "0000 000" findet hier nicht statt, weil `M+ E … E M-` folgt, s. 3.1 und 3.3)
3	"M" gedrückt halten	OR := OR OR 025 → BR := "0000 025" (Sprung mit Rückkehrabsicht, Adresse 0000)
4	1. "E"	Befehlsentnahme (Phase II) durch gehaltenes M verhindert; BZ := 0001, AR := 0001; ⟨AC⟩ := 0000 0002 000 (Rücksprungadresse BZ+1 im Adressfeld)
5	2. "E"	BZ := 0002, AR := 0002; ⟨AC⟩ := 0000 0003 000
6	3. "E"	BZ := 0003, AR := 0003; ⟨AC⟩ := 0000 0004 000
7	4. "E"	BZ := 0004, AR := 0004; ⟨AC⟩ := 0000 0005 000 (= 0000 0004 000 + 0000 0001 000)
8	"M" loslassen	OR-Verknüpfung mit 025 entfällt für Folgeschritte
9	H-R → "R"	Flag H := 0; Interpretation geht bei Konnektor B weiter, mit BZ = 0004 → nächster Befehl wird aus Adresse 0004 entnommen

3.3 Festlegungen zu den Tasten M, Ü und E (Stand 29.09.2026)

(Die Schreibweise der Token einer Tastenfolge steht im Dokument "C8205-Interpreter-Bedienpult_mit_Schaltern.md", Abschnitt "Tastenfolge-Token".)

E unter gehaltenem M (Vorgabe des Autors): Jedes E führt Phase IV (BR = ⟨AR⟩ 025, Sprung mit Rückkehrabsicht mit Löschen) und Phase I aus; Phase II ist gesperrt. Wirkung nach dem n-ten E (Start mit BZ = b): `BZ := b+n`, `AR := BZ`, und der AC enthält im Adressfeld (z_{12}–z_{23}) den Wert BZ+1, übrige Stellen 0 (das Löschbit von 025 ersetzt den AC, es wird nicht aufsummiert). Beispiel B (b=0, n=4): `⟨AC⟩ = 0000 0005 000`. Danach steht der Automat vor Phase II: ein nächstes `E` (nach Loslassen von M) beginnt bei Phase II (Knoten 8), ein `R` bei Konnektor B.

M hat nur zwei Stellungen: "M gedrückt" (`M+`) und "M losgelassen" (`M-`). Das Token `M` allein (Antippen) ist gleichbedeutend mit `M+` und sofort danach `M-`. Gedrücktes M muss innerhalb der Tastenfolge wieder losgelassen werden. Ungültig sind daher `M-` ohne vorheriges `M+`, ein zweites `M+` bei noch gedrücktem M, ein `R` bei noch gedrücktem M und eine Tastenfolge, die mit gedrücktem M endet. Wirkung: bei `M+` gilt `OR := OR ∨ 025`. Beim Loslassen gilt: Wurde unter gehaltenem M kein `E` betätigt (also beim Antippen `M`), bleibt BR = "⟨AR⟩ 025" aufgestellt und wird erst durch `E` oder `R` ausgeführt (Fall 4). Wurde unter M mindestens ein `E` betätigt (Beispiel B, Fall 6), entfällt der Beitrag von 025 beim Loslassen (übrige Anteile im OR, z. B. von Ü, bleiben).

Ergänzung 04.10.2026 (Originaldokumentation 4.3.2.3, Abb. 19): Der durch `M` aufgestellte BR gilt als Befehl, dessen Phase III bereits abgelaufen ist. Ist er ein Sprung mit Rückkehrabsicht (B=0 oder 1, A=2, Z≥4: 025, 027, 125, 127), beginnt die Ausführung deshalb nicht bei Knoten 19, sondern bei Konnektor 4 (Knoten 54). Die Sprungoperation selbst (Sichern der Rückkehradresse im AC, Knoten 40, ANWEISUNG-029) und die Löschanweisung entfallen. Die Umstellung auf unbedingten Sprung (A:=0, wie in Knoten 40) wird beim Eintritt vorweggenommen, damit Knoten 59 den Sprungteil über Phase IV führt. Es folgen der Überlauftest (Knoten 55), Phase IV nur mit Resultatbehandlung (Knoten 61 bis 63), die Koinzidenz W_1 (Knoten 7, 6, 5) und Phase II mit der Adresse in AR, also der nächste Befehl aus der Zelle, die im Adreßteil des Stop-Befehls steht. Jeder andere BR (z. B. nach zusätzlichem `Ü`) beginnt weiter bei Knoten 19.

Ü ist nur bei gesetztem Schalter H wirksam (wie alle Tasten außer ST). Bei nicht gesetztem H hat die Taste keine Wirkung. Bei gesetztem H gilt: `AR := 7777` (oktal, überschreibt unbedingt) und `OR := OR ∨ 240` (Transport); damit steht BR = "7777 240" (wenn OR vorher 000 war), aufgestellt und noch nicht ausgeführt. Ausgeführt wird BR erst durch `E` oder durch `R` (beides beginnt bei Knoten 19, Ende der Phase-III-Arbeit).

Entsteht BR aus dem Operationsteil eines Stop-Befehls (02Z, Z<4), ist das Ergebnis nicht Transport: 02Z ∨ 240 = 26Z, Konjunktion im Speicher mit negiertem AC, ⟨a⟩ := (¬AC) ∧ ⟨a⟩. Bei `Ü` steht in AR die Adresse 7777, Phase IV läuft also mit "7777 26Z" (Originaldokumentation 4.3.2.3, Abb. 19; Ergänzung 04.10.2026). Nur wenn OR vorher 000 war (z. B. nach `G`), entsteht 240 (Transport). Für den bedingten Stopp (06Z) ergibt dieselbe Rechnung ebenfalls 26Z (abgeleitet; das Original nennt nur den unbedingten Stopp).

ST ist die einzige Taste, die nicht an Schalter H gebunden ist, dafür aber elektronisch verriegelt: sie ist nur wirksam, wenn der Rechner im "Bedingten Stopp" (Flag BS) oder "Unbedingten Stopp" (Flag US) steht, sonst ohne

Wirkung. Wirkung: `OR := OR ∨ 240`; AR bleibt unverändert; damit ist BR = "⟨AR⟩ (⟨OR⟩ ∨ 240)" aufgestellt und noch nicht ausgeführt (Ausführung durch `E` oder `R`, Start bei Knoten 19); US wird dabei gelöscht. US entsteht in einer Tastenfolge durch `G` (Fall 1: `G` setzt US := 1) und wird durch `ST` oder `R` wieder gelöscht; ohne vorheriges `G` steht der Rechner zu Beginn einer Tastenfolge nicht im Stopp.

Ergänzung 04.10.2026: Nach einem Stop-Befehl (02Z bzw. 06Z, Z<4) entsteht durch `ST` die Operation 26Z (Konjunktion mit negiertem AC, s. Ü), nach `G` (OR = 000) dagegen 240. Die Fortsetzung verläuft wie bei der entstandenen Operation (Wartezeit W_2, Überlauftest, Phase IV, Phase I). Der Stopp selbst erfolgt vor der Wartezeit W_2 (Abb. 19).

BS und SP sind einrastende Schalter. `SP=1` (Schalter gedrückt und eingerastet) setzt das Flag SP auf 1, `SP=0` (Schalter gelöst) setzt es auf 0; mehr geschieht nicht. `BS=1` bzw. `BS=0` legt dagegen nur die Schalterstellung BS fest; das Flag BS ist nicht der Schalterzustand, es wird erst 1, wenn ein programmierter "Bedingter Stopp" bei eingerastetem Schalter wirksam wird. Da die Schalter mechanisch einrasten, kann `G` sie nicht lösen: `G` lässt das Flag SP und die Schalterstellung BS unverändert (wie H); das Flag BS löscht `G` (es beseitigt einen programmierten Stopp). Nur ein `BS=0` bzw. `SP=0` in der Tastenfolge löst die Schalter wieder. Die Schalter sind nicht an H gebunden und in jeder Reihenfolge zu den anderen Tasten zulässig.

E (ohne M) ist definiert (s. 4.2, Fall 5): Phase IV und Phase I, dann Phase II und III, danach Halt (Halt-Punkt HP3 = Eintritt in Knoten 19). E ist nur bei gesetztem Schalter H wirksam. Eine Tastenfolge darf ohne `R` enden; dann bleibt der Rechner im Halt und das Ergebnis der Sequenz ist der Zustand nach dem letzten Schritt. E unter gehaltenem M: s. 4.3 (Fall 6).

3a. Schritt 3 jedes Tests (optional): Ergebnisprüfung (`Prüfungen:`)

Ergänzung 30.09.2026.

Nach der Tastenfolge (Abschnitt 3) kann eine Testdatei zusätzlich einen dritten Abschnitt

```
Prüfungen:
```

enthalten. Er ist **optional**: fehlt er, endet der Test wie bisher ohne automatisierte Ergebnisprüfung. Ist er vorhanden, prüft der ausführende Interpreter (Python- wie Claude-Interpreter, s. Abschnitt 1a) nach Abschluss der interpretativen Abarbeitung den erreichten Endzustand gegen die hier angegebenen Soll-Werte und vermerkt im Protokoll `Test wahr` oder `Test falsch`. Soll ein Test bewusst `Test falsch` ergeben, steht hinter dem Abschnittskopf `False` (`Prüfungen: False`, Abschnitt 3a.5).

3a.1 Aufbau

Der Abschnitt `Prüfungen:` besteht aus höchstens zwei Teilen, in dieser Reihenfolge:

1. **Höchstens eine** Register-/Flag-Zeile. Sie enthält, durch Leerzeichen getrennt und in beliebiger Reihenfolge, eine oder mehrere Zuweisungen der Form `SCHLÜSSEL=WERT`. Zulässige Schlüssel sind ausschließlich die Register `BZ`, `AC`, `AR` sowie die Flags `FE`, `FÜ` (Schreibweise im Test: `FUE`), `BS`, `US`, `E`, `A`, `V1`, `V2` — also genau die Teilmenge, die auch in der abschließenden Zustandszeile des Interpreters ausgegeben wird (vgl. Abschnitt 8). Andere Register (z. B. OR) oder Flags (z. B. H, SP, T, SW, K1–K3) sind hier nicht vorgesehen. Die Zeile muss nicht vollständig sein: jeder der genannten Schlüssel darf, muss aber nicht angegeben werden; jeder darf höchstens einmal vorkommen.
   - `BZ=` und `AR=` je 4 Oktalstellen (Adressbreite, wie in Abschnitt 2 für Adressen festgelegt).
   - `AC=` 11 Oktalstellen (Maschinenwortbreite, wie in Abschnitt 2 für den Wert festgelegt).
   - Flag-Werte sind je genau eine Oktalstelle mit dem Wert `0` oder `1` (kein anderer Oktalwert zulässig).
2. **Beliebig viele** Speicherprüfzeilen, danach folgend, je eine pro Zeile, im selben Format wie die Adresse:Wert-Zeilen der Trommeldatei (Abschnitt 2): `Adresse: Wert` (Adresse 4, Wert 11 Oktalstellen). Sie legen fest, welchen Inhalt bestimmte Trommeladressen nach Testende haben müssen. Verglichen wird mit dem Trommelstand nach dem Lauf; bei Adressen, die der Test nicht selbst belegt, hängt der Istwert vom Trommelstand vor dem Test ab (Abschnitt 2c). Dieselbe Adresse darf nicht mehrfach mit widersprüchlichem Sollwert auftreten (wie bei den Trommeldatei-Zeilen selbst, s. Abschnitt 2/5).

Fehlt die Register-/Flag-Zeile (weil keine Register/Flags geprüft werden sollen), beginnt der Abschnitt unmittelbar mit Speicherprüfzeilen. Sind weder eine Register-/Flag-Zeile noch Speicherprüfzeilen angegeben (leerer `Prüfungen:`-Abschnitt), gilt das wie ein fehlender Abschnitt: keine Ergebnisprüfung.

Beispiel (wie oben skizziert):

```
Trommeldatei:
#DOK-0001 Test Ladebefehl und Addition
#PROT STUFE=2
0000: 00003000211
0001: 00003001210
0002: 00000001164
0003: 00000000020
# Beginn Datenbereich
3000: 00000000100
3001: 00000000045

Tastenfolge:
H G R

Prüfungen:
BZ=0003 AC=00000000045 AR=0003 FE=0 FUE=0 BS=0 US=1  E=0 A=0 V1=0 V2=0
0000: 00003000211
3000: 00000000100
3001: 00000000045
```

3a.2 Auswertung

Die Ergebnisprüfung wird erst nach Abschluss der interpretativen Abarbeitung der Tastenfolge und des sich ggf. anschließenden Knotenautomaten-Durchlaufs vorgenommen, also gegen den dann erreichten Endzustand (Register, Flags, Trommelinhalt).

Jede angegebene Register-/Flag-Zuweisung und jede angegebene Speicherprüfzeile wird einzeln mit dem tatsächlich erreichten Wert verglichen. Das Gesamtergebnis ist:

- **Test wahr:** ausschließlich dann, wenn **alle** angegebenen Einzelprüfungen (Register, Flags, Speicherzellen) mit dem tatsächlichen Endzustand übereinstimmen.
- **Test falsch:** sobald **mindestens eine** der angegebenen Einzelprüfungen nicht übereinstimmt.

Im Protokoll (bzw. auf dem Bildschirm, s. 3a.3) wird zu jeder angegebenen Einzelprüfung sowohl der Soll- als auch der Ist-Wert vermerkt, nicht nur das Gesamtergebnis — damit im Fehlerfall sofort erkennbar ist, welche Einzelprüfung(en) abweichen.

Test ohne ordnungsgemäßes Ende: Bricht die Abarbeitung vor Erreichen eines regulären Endzustands ab (Stub, Laufzeitfehler, Knotenlimit — s. Abschnitt 5, Grundsatz "laut scheitern"), wird keine Ergebnisprüfung vorgenommen; ein in diesem Fall dennoch vorhandener `Prüfungen:`-Abschnitt bleibt ohne Wirkung, es wird stattdessen vermerkt, dass die Ergebnisprüfung deswegen entfällt. Insbesondere gilt ein solcher Fall nicht automatisch als `Test falsch`.

Ausnahme Knotenlimit bei Ein-/Ausgabe: Erreicht der Lauf das Knotenlimit (2 000 000 Knoten) und wurde im laufenden oder im vorigen Befehl eine Ausgabe (ANWEISUNG-017) oder Eingabe (ANWEISUNG-032) ausgeführt, arbeitet der Rechner ein Gerät ab (Dauerausgabe, Warten auf Eingabe). Das gilt nicht als Fehler: Der Lauf endet ordnungsgemäß mit dem Grund „Knotenlimit erreicht bei aktivem Eingabe-/Ausgabe-Befehl“, die Ergebnisprüfung (3a) und die Dateiprüfungen (3b) werden wie üblich vorgenommen, das Gesamtergebnis ist `Test wahr`, wenn alle Prüfungen wahr sind (ohne Prüfungen: Rückgabecode 4). Im Protokoll (Abschnitt 2) werden die am Ende gleichen Zeilen nur einmal ausgegeben; davor steht eine Zeile „… n-mal dieselben p Zeile(n) weggelassen“. Ohne Ein-/Ausgabebefehl bleibt das Knotenlimit ein Fehler (Endlosschleife, Rückgabecode 8, Protokoll vollständig). Beispiel: `Tests/knotenlimit_ausgabeschleife.txt` (Endlosschleife mit Ausgabebefehl auf das Schreibwerk).

3a.3 Ausgabe

Die Ergebnisprüfung wird, unabhängig von der Protokollierungsstufe `#PROT STUFE=n` (Abschnitt 2, analog zur `#DOK`-Diagnose in Abschnitt 7), stets zusätzlich auf dem Bildschirm ausgegeben. Bei `#PROT STUFE=2` wird sie außerdem am Ende der Protokolldatei (Abschnitt 8) vermerkt, im Anschluss an die dortige Endzustandszeile.

3a.4 Formale Gültigkeitsprüfung

Der `Prüfungen:`-Abschnitt unterliegt, wie Trommeldatei und Tastenfolge, der formalen Gültigkeitsprüfung vor Testbeginn (s. Abschnitt 5): unbekannte Schlüssel in der Register-/Flag-Zeile, ein Schlüssel mit falscher Stellenzahl oder unzulässigem Wert, ein mehrfach angegebener Schlüssel, eine Speicherprüfzeile außerhalb des Adresse:Wert-Formats sowie eine widersprüchlich mehrfach belegte Prüfadresse führen dazu, dass der Test **nicht startet** (Grundsatz "laut scheitern", keine eigenmächtige Korrektur). Für die Kopfzeile des Abschnitts gilt Abschnitt 3a.5.

3a.5 Erwartetes Ergebnis „falsch“: `Prüfungen: False` (Ergänzung 10.10.2026)

Manche Tests sollen bewusst `Test falsch` ergeben (Negativtests, die zeigen, dass eine Abweichung erkannt wird). Der Rückkehrcode 6 würde in den Sammelläufen als Fehler gemeldet. Deshalb darf hinter dem Abschnittskopf **optional** `False` stehen:

```
Prüfungen: False
BZ=0003 AC=00000000046 AR=0003
3001: 00000000045
```

Schreibweise. Die Kopfzeile lautet `Prüfungen:`, danach höchstens Leerzeichen und das Wort `False` (Groß- und Kleinschreibung unerheblich), sonst nichts. Der übrige Abschnitt (Register-/Flag-Zeile, Speicherprüfzeilen, Dateiprüfungen nach Abschnitt 3b) ist unverändert (3a.1) und wird wie sonst ausgewertet und protokolliert. `Prüfungen:` ohne `False` gilt wie bisher.

Wirkung im Interpreter. Die Auswertung ändert sich nicht: Das Gesamtergebnis (`Test wahr` / `Test falsch`, 3a.2 und 3b.2) und der Rückkehrcode (0 bzw. 6) bleiben, wie sie sind. Der Interpreter ergänzt nur eine Vermerkzeile hinter dem Gesamtergebnis (Bildschirm, bei `#PROT STUFE=2` auch Protokolldatei): `Erwartetes Ergebnis: falsch (Prüfungen: False)`.

Wirkung in den Skripten. Die Skripte erkennen die Kopfzeile in der Testdatei, die an den Interpreter übergeben wird, an dem Muster `^Prüfungen:[[:space:]]*[Ff][Aa][Ll][Ss][Ee][[:space:]]*$` (ohne Ordner wird zusätzlich in `Tests/` gesucht). Der Rückkehrcode bleibt in allen Fällen 6; der Lauf gilt als **Test wahr** und macht keine Ausgabe:

- `c8205_bis_test.sh` startet den eigentlichen Test (Schritt 2) mit `--NO-TERMOUT`. Am Bildschirm erscheinen dann weder die Einzelprüfzeilen mit `FEHLER` noch `Test falsch` noch die Vermerkzeile; die Ausgaben des C8205-Programms selbst und Meldungen der Vorab-Prüfung bleiben (Python_Interpreter.md, Abschnitt 5.1). Das Protokoll bei `#PROT STUFE=2` bleibt vollständig. Das Laden des Minimalprogramms (Schritt 1) ist nicht betroffen.
- `compute_all_tests.sh` startet alle Tests ohnehin mit `--NO-TERMOUT` und meldet Code 6 bei einer solchen Testdatei nicht (wie die Codes 0 und 4).

Grenzen.

- Alle anderen Rückkehrcodes (2, 8, 16) werden auch bei `Prüfungen: False` wie bisher behandelt; `compute_all_tests.sh` meldet sie weiter.
- Ein Test mit `Prüfungen: False` ohne jede Prüfzeile hat keine Ergebnisprüfung (Code 4); `False` bleibt ohne Wirkung.
- Endet der Test nicht ordnungsgemäß (3a.2), entfällt die Ergebnisprüfung samt Vermerk.

Formale Gültigkeit. Jeder andere Text hinter `Prüfungen:` (auch `True`) ist ein Formfehler: Der Test startet nicht (Rückkehrcode 16, 3a.4).

3b. Dateiprüfungen für Stanzer und Schreibwerk (`--pruefe-stanzer1`, `--pruefe-sw-ausgabe`)

Ergänzung 03.10.2026.

Die Ergebnisprüfung nach Abschnitt 3a prüft Register, Flags und Trommelinhalt. Programme, die auf Kanal K1 (Lochstreifenstanzer, 6.3.2) oder K2 (Schreibwerk, 6.3.1) ausgeben, erzeugen zusätzlich eine Ausgabedatei (`stanzer-ausgabe.txt` bzw. `sw-ausgabe.txt`). Zwei weitere, ebenfalls optionale Prüfungen vergleichen diese Dateien mit je einer Soll-Datei. Sie werden als **Zeile im Abschnitt** `Prüfungen:` **der Testdatei** angegeben (nach der Register-/Flag-Zeile oder zwischen den Speicherprüfzeilen, je Zeile eine Prüfung, jede höchstens einmal) und stehen damit beim Test. Zusätzlich kann der Python-Interpreter sie als Schalter der Kommandozeile erhalten; ein Schalter hat Vorrang vor der Zeile der Testdatei. Beim Claude-Interpreter gilt die entsprechende virtuelle Zuordnung bei der Testdefinition:

Schalter	Soll-Datei wird verglichen mit
`--pruefe-stanzer1 DATEI`	der Datei, die der Lauf über `--stanzer1 PFAD` (Schalter **oder** Zeile der Testdatei, s. 2b) erzeugt hat (bei einem Verzeichnis: `stanzer-ausgabe.txt` darin)
`--pruefe-sw-ausgabe DATEI`	der Datei, die der Lauf über `--sw-ausgabe PFAD` (Schalter **oder** Zeile der Testdatei, s. 2b) erzeugt hat (bei einem Verzeichnis: `sw-ausgabe.txt` darin)

Beide Prüfungen können einzeln oder gemeinsam angegeben werden, auch zusammen mit Register-, Flag- und Speicherprüfungen. Ein `Prüfungen:`-Abschnitt, der nur Dateiprüfungen enthält, ist gültig.

Gerätezuordnung in der Testdatei: Auch `--sw-ausgabe PFAD`, `--stanzer1 PFAD`, `--leser1 PFAD`, `--leser2 PFAD` und `--k0 datei=PFAD` dürfen als eigene Zeile in der Testdatei stehen (Abschnitt 2b). Eine Schreibwerk-Prüfung lässt sich damit vollständig in der Testdatei beschreiben: die Zeile `--sw-ausgabe PFAD` im Abschnitt `Trommeldatei:` und die Zeile `--pruefe-sw-ausgabe DATEI` im Abschnitt `Prüfungen:`. Ebenso lässt sich eine Stanzer-Prüfung vollständig in der Testdatei beschreiben: die Zeile `--stanzer1 PFAD` im Abschnitt `Trommeldatei:` und die Zeile `--pruefe-stanzer1 DATEI` im Abschnitt `Prüfungen:`.

3b.1 Soll-Datei und Ist-Datei

- **Soll-Datei** ist die hinter dem Schalter genannte `DATEI`. Ein reiner Dateiname ohne Verzeichnisangabe liegt im Verzeichnis der Testdatei (üblicherweise `Tests/`); eine Angabe mit Verzeichnis gilt so, wie sie geschrieben ist (relativ zum Arbeitsverzeichnis). Die Soll-Dateien liegen im Ordner `Test-Ergebnisvergleiche/` und werden in der Testdatei mit diesem Pfad genannt. Beispiel: Der Test `Tests/bsp_seite_42_025_stanzer.test` enthält die Zeile `--pruefe-stanzer1 Test-Ergebnisvergleiche/bsp_seite_42_025_stanzer-stanzer-ausgabe.txt`; verglichen wird mit der Datei `Test-Ergebnisvergleiche/bsp_seite_42_025_stanzer-stanzer-ausgabe.txt`. **Namenskonvention (Ergänzung 08.10.2026):** Soll-Dateien sind keine Tests und liegen deshalb nicht in `Tests/`. Ihr Name endet auf `-ausgabe.txt` oder `_erwartet.txt`; `compute_all_tests.sh` überspringt Dateien mit solchen Namen. Läge eine Soll-Datei mit anderem Namen in `Tests/`, würde das Skript sie als Test starten (Rückkehrcode 16, weil Tastenfolge und Zeilenformat fehlen).
- **Ist-Datei** ist die vom Lauf erzeugte Ausgabedatei des jeweiligen Geräts (6.3.1, 6.3.2), z. B. `Test-virtuelleGeraete/Stanzer1/stanzer-ausgabe.txt`. Maßgeblich ist nur eine Datei, die **dieser Lauf** angelegt hat: eine aus einem früheren Lauf übrig gebliebene Datei gilt als nicht vorhanden, wenn der Lauf selbst keine Ausgabe auf dem Gerät vorgenommen hat.

3b.2 Auswertung

Der Vergleich erfolgt wie bei einem `diff` zeilenweise nach Abschluss der interpretativen Abarbeitung:

- **Prüfung falsch**, wenn die Soll-Datei fehlt, **oder** die Ist-Datei fehlt, **oder** sich beide Dateien im Inhalt unterscheiden. Fehlen beide, ist die Prüfung ebenfalls falsch.
- **Prüfung wahr**, wenn beide Dateien vorhanden und zeilenweise gleich sind. Unterschiede nur im Zeilenende (CR LF gegen LF) oder ein fehlendes Zeilenende nach der letzten Zeile gelten nicht als Unterschied; alle anderen Abweichungen, auch Leerzeichen am Zeilenende, schon.

Das Gesamtergebnis `Test wahr` / `Test falsch` (3a.2) ergibt sich aus **allen** angegebenen Prüfungen gemeinsam: `Test wahr` nur, wenn jede Einzelprüfung nach 3a und jede Dateiprüfung nach 3b wahr ist. Wie bei 3a entfallen die Dateiprüfungen, wenn der Test nicht ordnungsgemäß endet (Stub, Laufzeitfehler, Knotenlimit); das ist dann nicht automatisch `Test falsch`. Die Kopfzeile `Prüfungen: False` (3a.5) gilt für das Gesamtergebnis aus 3a und 3b gemeinsam.

3b.3 Ausgabe

Wie 3a.3: stets auf dem Bildschirm (soweit nicht unterdrückt), bei `#PROT STUFE=2` zusätzlich in der Protokolldatei (Abschnitt 8.1) im Anschluss an die Einzelprüfungen aus 3a. Je Dateiprüfung steht eine Zeile mit Soll- und Ist-Pfad und `OK` bzw. `FEHLER` samt Grund (`Soll-Datei fehlt`, `Ist-Datei fehlt, vom Lauf nicht angelegt` oder `Inhalt verschieden`); bei verschiedenem Inhalt folgt der `diff` (Soll gegen Ist, höchstens 100 Zeilen, danach ein Hinweis auf die Restzahl).

3b.4 Formale Gültigkeitsprüfung

Die Dateiprüfungen unterliegen der Prüfung vor Testbeginn (Abschnitt 5): `--pruefe-stanzer1` ist nur zusammen mit `--stanzer1` (als Schalter oder als Zeile der Testdatei, Abschnitt 2b), `--pruefe-sw-ausgabe` nur zusammen mit `--sw-ausgabe` (als Schalter oder als Zeile der Testdatei, Abschnitt 2b) zulässig, weil sonst keine Ist-Datei existieren kann; andernfalls startet der Test nicht. Als Schalter der Kommandozeile sind beide außerdem nur bei **einer** Testdatei je Aufruf zulässig. Eine Zeile ohne Dateiname oder eine mehrfach angegebene Dateiprüfung in der Testdatei verhindert den Teststart wie jeder andere Formfehler des `Prüfungen:`-Abschnitts (3a.4). Eine fehlende Soll-Datei ist dagegen kein Formfehler, sondern das Prüfergebnis `falsch` (3b.2).

Beispiel:

Ende der Testdatei `Tests/bsp_seite_42_025_stanzer.test`:

```
Prüfungen:
BZ=2014 AC=00002012000 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
2012: 00000001164
...
2022: 00000000777
--pruefe-stanzer1 Test-Ergebnisvergleiche/bsp_seite_42_025_stanzer-stanzer-ausgabe.txt
```

Aufruf:

```
./c8205_interpreter.sh --NO-TERMOUT --stanzer1 Test-virtuelleGeraete/Stanzer1/ bsp_seite_42_025_stanzer.test
```

Der Test stanzt drei Zeichen nach `Test-virtuelleGeraete/Stanzer1/stanzer-ausgabe.txt`. Verglichen wird diese Datei mit `Test-Ergebnisvergleiche/bsp_seite_42_025_stanzer-stanzer-ausgabe.txt`. Ist die Datei mit den Soll-Zeilen `00110001`, `00110010`, `00100011` identisch und stimmen auch die `Prüfungen:` des Tests, endet der Lauf mit `Test wahr` (Rückkehrcode 0), sonst mit `Test falsch` (Rückkehrcode 6).

4. Weitere typische Tastensequenzen (zur Illustration, nicht Teil des Test-Starts)

Die folgenden Sequenzen sind **unabhängig voneinander**: jede führt nur die Wirkung ihrer eigenen, tatsächlich aufgeführten Tasten aus. Es gibt **keinen** gemeinsamen oder anderweitig spezifizierten Vor-Zustand — nicht genannte Register/Flags bleiben von der jeweiligen Sequenz unberührt und werden mit ihrem (unspezifizierten) Vorwert notiert (z. B. ⟨AR⟩, ⟨OR⟩, ⟨BZ⟩, ⟨AC⟩).

4.1 Beseitigen der Betriebszustände US, BS, FÜ und FE

Fall 1 — H, G

Aktion	Wirkung
H	H:=1
G	alle Flags außer H, T, SP := 0 (damit u. a. auch FÜ:=0, FE:=0, BS:=0, falls zuvor gesetzt; SP ist ein eingerasteter Schalter und bleibt unverändert, s. 3.3); zusätzlich wird **US:=1** gesetzt (Voraussetzung für "G" ist der Halt-Zustand, und "G" stellt BR auf "0000 000"); BR:="0000 000" (AR:=0000, OR:=000); K0 angewählt, T:=1; BZ unverändert

(Anmerkung: Ob die Hardware nach "H, G" tatsächlich US=1 anzeigt – etwa weil der zuvor programmierte Stopp-Befehl formal noch nicht abgearbeitet ist – oder ob G stattdessen auch US auf 0 setzt, ist in der Originaldokumentation nicht ausdrücklich belegt und bleibt offen. Wirkungslos ist es in jedem Fall: Der Automat steht durch H, und "R" überwindet US unabhängig von seinem Wert.)

Kein R in der Sequenz → Automat bleibt in H stehen. Programm ist unterbrochen.

Fall 2 — ST

Aktion	Wirkung
ST	nur wirksam, wenn Zustand "Bedingter Stopp" oder "Unbedingter Stopp" vorliegt (elektronische Verriegelung); dann: OR := ⟨OR⟩ OR 240 → BR := ⟨AR⟩ (⟨OR⟩ OR 240); AR bleibt unverändert; US := 0; BR aufgestellt, noch nicht ausgeführt (Start bei Knoten 19 durch E oder R)

H wird durch ST nicht beeinflusst (ST ist die einzige der hier genannten Tasten, die nicht an Stellung H gebunden ist).

Fall 3 — H, Ü, R

Aktion	Wirkung
H	H:=1
Ü	nur wirksam, weil H gesetzt ist (ohne H: keine Wirkung); AR:="7777" (überschreibt unbedingt); OR := ⟨OR⟩ OR 240 → BR:="7777 (⟨OR⟩ OR 240)"
R	H:=0; Interpretation setzt bei Phase IV fort → Ausführung von BR

Fall 4 — H, M, R (M ist hier gleichbedeutend mit M gedrückt und sofort M losgelassen, s. 3.3)

Aktion	Wirkung
H	H:=1
M	AR bleibt unverändert (⟨AR⟩); OR := ⟨OR⟩ OR 025 → BR:="⟨AR⟩ (⟨OR⟩ OR 025)"; kein E unter M, daher bleibt BR aufgestellt
R	H:=0; Interpretation setzt bei Phase IV fort: bei Sprung mit Rückkehrabsicht (025, 027) ab Konnektor 4 (Knoten 54), sonst ab Knoten 19 → Ausführung von BR (Ergänzung 04.10.2026, s. unten)

Die Zuordnung zu "US/BS wird übergangen" (Dokumenttext) ist keine aus der Tastenfolge selbst ableitbare Tatsache, sondern gilt nur unter der Zusatzannahme des jeweiligen Anwendungsfalls, dass AR/OR zufällig gerade den US-Befehl enthalten. Das gehört zur Anwendung, nicht zur Definition der Sequenzen.

Wirkung nach Originaldokumentation 4.3.2.3 (Ergänzung 04.10.2026, Abb. 19): Nach einem unbedingten Stopp (A=2, Z<4) entsteht durch OR ∨ 025 die Operation 025 oder 027. Da Phase III des Stop-Befehls bereits abgelaufen ist, wird von dieser Operation nur die Resultatbehandlung ausgeführt (Linksverschiebung des AC, bei 027 zyklisch). Die Sprung- operation selbst (⟨AC⟩ := Vr⟨BZ⟩ ∨ ⟨AC⟩) und die Löschanweisung gehören zu Phase III und entfallen. Anschließend wird der nächste Befehl aus der Speicherzelle entnommen, die im Adreßteil des Stop-Befehls angegeben ist (Phase II mit AR). Knotenfolge: 54, 55, 59, 61, 62, 63, 7, 6, 5, 8 (Einstieg s. 3.3). ⟨BZ⟩ wird erst in Phase II (Knoten 11) und Phase III (Knoten 16) neu gesetzt.

Übertragung auf BS, FÜ, FE (lt. Dokument "C8205-Interpreter-Bedienpult_mit_Schaltern.md"):

- **BS**: verhält sich wie US, alle vier Fälle anwendbar. Zusätzlich beseitigt auch das Lösen des Schalters "BS" allein den Stopp: der Ablauf geht dann wie bei nicht erfüllter Schalterbedingung weiter (Überlauftest, eine Wortzeit, Phase I; Originaldokumentation Abb. 19, Ergänzung 04.10.2026). Bei Fall 4 wird der nächste Befehl abgearbeitet (06Z ∨ 025 ergibt den bedingten Sprung mit Rückkehrabsicht 065/067, der ohne Phase IV und ohne Verschiebung mit Phase I weiterläuft, Knoten 24, 55, 60).
- **FÜ**: nur Fall 1 (H, G) anwendbar; zusätzlich ein eigener Fall "H; E gedrückt halten; G; E loslassen", der nur den Überlauf löscht statt aller Flags/BR.
- **FE**: nur durch G aufhebbar (entspricht Fall 1, H, G).

4.2 Transport des Akkumulatorinhaltes nach Speicherplatz 7777

Fall 5 — H, G, Ü, E (E ist hier ein einzelner Tipp, E ohne M; Ü wirkt, weil H gesetzt ist)

Aktion	Wirkung
H	H:=1
G	alle Flags außer H,T,SP := 0, danach US:=1; BR:="0000 000" (AR:=0000, OR:=000); K0 angewählt, T:=1; BZ unverändert (⟨BZ⟩)
Ü	AR:="7777"; OR := 000 OR 240 = 240 → BR:="7777 240"
E (einzelner Tipp)	Phase IV: Ausführung von BR="7777 240" → <7777>:=<AC>. Danach Phase I: <AR>:=<BZ> (=⟨BZ⟩). Danach Phase II: <BZ>:=ADR; <OR>:=OP (Inhalt der Zelle ⟨BZ⟩). Danach Phase III: <AR>:=<BZ>=ADR, gleichzeitig <BZ>:=<AR>+1. Automat stoppt danach wieder (H bleibt =1)

Ergebnis: <7777> := <AC>, danach Stopp mit neu geladenem BR aus Zelle ⟨BZ⟩.

4.3 Transport des Befehlszählerinhaltes nach dem Akkumulator

Fall 6 — H, G, M-gedrückt-halten, E, M-loslassen, E

Aktion	Wirkung
H	H:=1
G	alle Flags außer H,T,SP := 0, danach US:=1; BR:="0000 000" (AR:=0000, OR:=000); K0 angewählt, T:=1; BZ und AC bleiben unverändert (⟨BZ⟩, ⟨AC⟩ — G hat keinen Einfluss auf Akkumulator- und Befehlszählerinhalt)
M gedrückt halten	OR := 000 OR 025 = 025 → BR:="0000 025" (Sprung mit Rückkehrabsicht); solange M gehalten wird, verhindert ein nachfolgendes E die Befehlsentnahme in Phase II
1. E (unter gehaltenem M)	Phase IV wird ausgeführt: BR="0000 025" → <AC> erhält die um 2 erhöhte Adresse ⟨BZ⟩+2 des nächsten abzuarbeitenden Befehls (im Adressfeld, s. 3.3); BZ := ⟨BZ⟩+1, AR := BZ; Phase II bleibt wegen gehaltenem M gesperrt, Automat steht vor Phase II
M loslassen	Sperre der Phase-II-Verhinderung entfällt
2. E (einzelner Tipp)	Phase II und Phase III werden durchlaufen; danach stoppt die Abarbeitung wieder (kein R in der Sequenz, H bleibt =1)

Ergebnis: `<AC> := ⟨BZ⟩+2`.

5. Formale Gültigkeitsprüfung vor Testbeginn

Vor jedem Test sind die zum Test angegebenen Ausgangsbedingungen (Trommeldatei gemäß Abschnitt 2, Tastenfolge gemäß Abschnitt 3, ggf. mit den Anfangswerten `#AC=`/`#BZ=`) sowie die Debug-Anforderungen (Testmarken `#DOK-xxxx`, `#SPEI-xxxx` und `#SPUR-xxxx`, s. Abschnitt 2) formal auf Gültigkeit zu prüfen. Der Test startet nur, wenn diese Prüfung insgesamt positiv verläuft; andernfalls startet der Test nicht.

Zu prüfen ist dabei insbesondere:

- **Trommeldatei:** Einhaltung des in Abschnitt 2 festgelegten Zeilenformats (Adresse 4 Oktalstellen, Wert 11 Oktalstellen, ausschließlich Oktalzahlen, zulässige Kommentar- und Leerzeilen); zusätzlich: keine Adresse ist mehrfach mit widersprüchlichem Wert belegt (jede Adresse hat innerhalb eines Tests genau einen definierten Speicherinhalt).
- **Persistente Trommel (Python-Interpreter, s. Abschnitt 2c):** Die Trommeldatei `trommel.txt` muss vorhanden und beschreibbar sein und genau 4096 Zeilen `aaaa: tttttttttt` mit lückenlos aufsteigenden Adressen `0000` bis `7777` enthalten. Diese Prüfung gilt für den ganzen Aufruf und läuft vor dem ersten Test; schlägt sie fehl, startet kein Test. Ausnahme: Steht `#TROMMEL=LEER` im ersten Test des Aufrufs, wird die Trommeldatei neu angelegt (Abschnitt 2g).
- **Testmarken** `#SPEI-xxxx`**:** Eine solche Testmarke ist nur zu einem im selben Test bereits definierten `#DOK-xxxx` mit gleichem `xxxx` zulässig; die angegebene Adressspanne `ssss-eeee` muss eine gültige Spanne bilden (s. dazu den Hinweis in Abschnitt 2 zum dortigen mutmaßlichen Tippfehler).
- **Testmarke** `#PROT STUFE=n`**:** `n` ist eine der Ziffern 0, 1 oder 2; die Marke kommt höchstens einmal vor. Fehlt sie, ist das gültig (Standardwert STUFE=1).
- **Testmarken** `#AC=` **und** `#BZ=` **(Ergänzung 08.10.2026):** jede höchstens einmal; der Wert besteht aus genau 11 (`#AC=`) bzw. 4 (`#BZ=`) Oktalziffern (0 bis 7). Fehlt eine Marke, ist das gültig (Register startet mit 0, Abschnitt 2).
- `#STARTE` **(Ergänzung 09.10.2026, Abschnitt 2e):** Wert nicht leer, 1 bis 4 Oktalziffern, höchstens einmal, nicht im Abschnitt `Prüfungen:`, keine zusätzliche Tastenfolge; ein Test mit `#STARTE` braucht keinen Abschnitt `Tastenfolge:`.
- `#TROMMEL=` **(Ergänzung 13.10.2026, Abschnitt 2g):** Wert `LEER` oder `MP`, höchstens einmal, nicht im Abschnitt `Prüfungen:`; bei `MP` muss die Trommelvorlage `Trommel/Trommelvorlagen/MP.txt` vorhanden und formal gültig sein (Format wie die Trommeldatei, 2c). Fehlt die Marke, ist das gültig (die Trommel bleibt unverändert).
- `#INTERRUPT` **(Ergänzung 08.10.2026, Abschnitt 2d):** nur im Abschnitt `Trommeldatei:`, zwischen zwei Befehlszeilen, höchstens einmal je Befehlszeile; erste Taste `H`, nur bekannte Tasten, zulässige M-Folge, mindestens eine Taste.
- **Tastenfolge:** Sie definiert, wie in Abschnitt 3 beschrieben, den Ausgangszustand des Rechners vor Eintritt in Konnektor B vollständig; eine Tastenfolge, aus der sich kein vollständig definierter Ausgangszustand ergibt, ist nicht gültig. Ungültig sind insbesondere (s. 3.3): M losgelassen ohne vorheriges M gedrückt, M gedrückt bei

bereits gedrücktem M, R oder Ende der Tastenfolge bei noch gedrücktem M, unbekannte Tasten sowie ein Ende weder mit `R` noch im Halt (H gesetzt). Ü und E bei nicht gesetztem H sind gültig, aber ohne Wirkung.

- **Abschnitt** `Prüfungen:` **(optional, s. Abschnitt 3a):** sofern vorhanden, müssen die Register-/ Flag-Zeile (zulässige Schlüssel, Stellenzahl, Flag-Wert 0/1, keine Mehrfachnennung eines Schlüssels) und die Speicherprüfzeilen (Adresse:Wert-Format, keine widersprüchlich mehrfach belegte Prüfadresse) den in Abschnitt 3a.1/3a.4 festgelegten Vorgaben entsprechen. Hinter `Prüfungen:` ist nur `False` zulässig (3a.5).
- **Geräte-Zeilen (optional, s. Abschnitt 2b):** `--leser1`, `--leser2`, `--stanzer1`, `--sw-ausgabe` und `--k0` jeweils mit Wert und höchstens einmal in der Testdatei; `--k0` in der Form `datei=PFAD`; die Pfade von `--k0`, `--leser1` und `--leser2` müssen vorhanden sein. Eine Zeile `--SW-TERMINAL` ist stets ein Formfehler (nur Kommandozeile, 2b und 6.3.1).
- **Dateiprüfungen (optional, s. Abschnitt 3b):** `--pruefe-stanzer1` setzt `--stanzer1`, `--pruefe-sw-ausgabe` setzt `--sw-ausgabe` voraus; beide nur bei einer Testdatei je Aufruf (3b.4).

Ergibt die formale Prüfung eine Unstimmigkeit – sei es beim Zeilenformat, bei einer widersprüchlich mehrfach belegten Adresse, einer ungültigen `#SPEI-xxxx`-Spanne oder einer unvollständig definierenden Tastenfolge –, so darf der Test **nicht eigenmächtig korrigiert und gestartet werden**. Die Durchführung wartet in diesem Fall auf eine Klärung mit dem Auftraggeber; erst nach dieser Klärung (ggf. mit korrigierten Ausgangsdaten) kann der Test regulär gestartet werden.

> **Grundsatz "laut scheitern":** Ergibt sich bei der Durchführung eines Tests – sei es bei der formalen Prüfung (s. o.) oder später bei der Abarbeitung des Knotengraphen – eine Unklarheit, ein Widerspruch oder eine fehlende Festlegung, wird dies **laut gemeldet und der Test angehalten**, statt eigenmächtig eine Annahme zu treffen und stillschweigend fortzufahren. Dieser Grundsatz gilt unabhängig davon, ob der Test durch den Python-Interpreter oder durch eine interpretierende KI (s. Abschnitt 1a) durchgeführt wird.

6. Interaktive Zeichenein- und -ausgabe während der interpretativen Abarbeitung

6.0 Kanalanwahl: Bedienpult-Tasten und Anwahlbefehl

Die Kanäle K0 bis K3 können auf zwei Arten ausgewählt werden:

- durch die **Bedienpult-Tasten K0–K3** (rastend; Teil der Tastenfolge zur Definition des Ausgangszustands, s. Abschnitt 3; Wirkung s. Datei "C8205-Interpreter-Bedienpult_mit_Schaltern.md": setzt das Flag des gewählten Kanals auf 1, die Flags der anderen drei auf 0; bei K0 zusätzlich T oder SW je nach Schalter SW),
- durch einen **Anwahlbefehl** im Programm selbst (`4700 030` = K0, `4600 030` = K1, `4500 030` = K2, `4400 030` = K3).

Ein zur Laufzeit ausgeführter Anwahlbefehl **übersteuert** die zuvor per Taste eingestellte Kanalstellung. Beide Interpreter (Claude- und Python-Interpreter) können über die Test-Anforderungs- Definition (Tastenfolge, Abschnitt 3) eine **initiale** Kanalanwahl erhalten. Kanal K3 bewirkt in dieser Ausbaustufe nichts (kein Gerät zugeordnet).

6.1 Kanal K0: Eingabe (Rechnerverhalten und Eingabequelle sind unabhängig)

Für Kanal K0 sind zwei voneinander unabhängige Dinge zu unterscheiden:

- **Rechnerverhalten (Flags T/SW):** K0 ist ein einziger Eingabekanal mit den zwei Subkanälen T (Tastatur des Bedienpultes) und SW (Schreibwerk). Welcher der beiden Subkanäle aktiv ist (Flag T oder Flag SW), entscheidet ausschließlich der Schalter SW im Moment des Anwählens von K0 (s. Datei "C8205-Interpreter-Bedienpult_mit_Schaltern.md", Abschnitt "Schalter 'SW'"). Dies ist reine Geräte-/Flaglogik, unabhängig von der folgenden Eingabequelle.
- **Eingabequelle für den Interpreter:** bestimmt, woher der Interpreter das tatsächliche Zeichen für einen auf K0 anstehenden Eingabebefehl nimmt – unabhängig davon, ob dabei Flag T oder Flag SW gesetzt ist (s. 6.1.1/6.1.2).

6.1.1 Standardfall: Eingabe über die Host-Tastatur

Erreicht die interpretative Abarbeitung des Knotenautomaten einen Eingabebefehl ("Eingabe-Halt", vgl. Flag E, Definition Datei "C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md"), bei dem Kanal K0 angewählt ist, so wird die Eingabe im Standardfall – anders als bei den Lochstreifenlesern (Kanäle K1/K2, dort Eingabe aus der jeweiligen, in Abschnitt 6.2 beschriebenen Lochstreifendatei) – nicht aus einer Datei entnommen, sondern über die Tastatur des Host-Systems angefordert und entgegengenommen. Das gilt für T und SW gleichermaßen.

Ablauf:

1. Der Interpreter stellt beim Erreichen des Eingabebefehls fest, dass K0 angewählt ist.
2. Die den Test durchführende KI (Claude bzw. die jeweils interpretierende KI) fordert daraufhin auf dem Host-Bildschirm die Eingabe eines Zeichens an und stellt diese Anforderung dar.
3. Das über die Host-Tastatur eingegebene Zeichen wird entgegengenommen und gemäß der geltenden Zeichenkode-Tabelle (Datei "C8205-Interpreter-Kodetabellen.md") in die entsprechende Eingabeverschlüsselung umgesetzt – analog zur Behandlung der physischen Eingabetasten (s. Datei "C8205-Interpreter-Bedienpult_mit_Schaltern.md", Abschnitt "Eingabetasten": disjunktive Verknüpfung mit dem Adressregister, danach Aufhebung der Sperre durch Umstellung des Eingabebefehls).
4. Die Eingabe erfolgt zeichenweise: Pro zu interpretierendem Eingabebefehl wird genau eine Host-Tastatur-Eingabe angefordert und entgegengenommen – nicht etwa eine ganze Zeile oder mehrere Zeichen auf einmal.

Hinweis auf wartende Terminal-Eingabe (Ergänzung 13.10.2026, Python-Interpreter): Wartet der Interpreter auf ein Zeichen vom Host-Terminal, ohne dass K0 über `--k0` umgeleitet ist (Standardfall dieses Abschnitts), so steht das Terminal sonst scheinbar still. Deshalb schreibt der Interpreter beim **ersten** solchen Warten eines Tests einmal die Zeile

```
--- Achtung: Terminaleingaben erforderlich (K0 ohne Umleitung --k0); Abbruch mit Strg+C ---
```

auf den Fehlerkanal (stderr; gelb, wenn stderr ein Terminal ist und die Umgebungsvariable `NO_COLOR` nicht gesetzt ist). Dazu gilt:

- Die Zeile erscheint nur einmal je Test, nicht vor jedem Eingabe-Halt.
- Sie erscheint nur, wenn der Standardeingang ein Terminal ist. Kommt die Eingabe über eine Pipe oder eine Dateiumleitung (<), wartet niemand am Terminal, und es erscheint nichts.
- Sie erscheint auch mit `--NO-TERMOUT`, weil sie keine Ergebnisausgabe ist.
- Ist K0 mit `--k0` umgeleitet (6.1.2), erscheint sie nie.
- Strg+C beendet den Test.
- Stand 24.10.2026: Die Tests `Tests/pj2-bis_v6_*.test` tragen die Tastenfolge H G R und warten sonst auf Tastatureingabe; `compute_all_tests.sh` startet sie deshalb mit geschlossenem Standardeingang (`</dev/null`), `c8205_bis_test.sh` lädt das Minimalprogramm mit `--LOADONLY`.

6.1.2 Flag für Datei-Eingabe: beide Subkanäle aus Datei

Wird dem Interpreter für K0 eine Eingabedatei bzw. ein Eingabeverzeichnis zugeordnet (Python-Interpreter: Schalter `--k0 datei=PFAD` oder gleichlautende Zeile in der Testdatei, s. 2b; Claude-Interpreter: entsprechend virtuell bei der Testdefinition), so werden abweichend von 6.1.1 die Eingaben für **beide** Subkanäle (T und SW) aus dieser Quelle entnommen, nicht vom Terminal.

- `PFAD` bezeichnet ein Verzeichnis, in dem eine oder mehrere Dateien liegen können; liegen mehrere gleichzeitig vor, werden sie in alphanumerischer Reihenfolge verarbeitet (analog 6.2.2/6.2.3 bei den Lochstreifenlesern).
- Existiert der angegebene Pfad nicht, ist dies (beim Python-Interpreter) ein Fehler, der bereits bei der formalen Vorab-Prüfung (Abschnitt 5) erkannt und gemeldet wird; der Test wird dann nicht gestartet.
- Ist der Pfad vorhanden, aber die aktuelle Datei zu Ende bzw. keine Datei (mehr) vorhanden, während ein Eingabebefehl auf K0 ansteht, wartet der Interpreter still (ohne Meldung), bis eine neue Datei in den Pfad kopiert bzw. beim Claude-Interpreter hochgeladen wird – wie bei den Lochstreifenlesern (6.2).

Fehlt dieses Flag, gilt der Standardfall aus 6.1.1 (Terminal-Eingabe, mit dem dortigen Hinweis).

6.1.3 Einordnung

Diese Festlegungen gelten für die interpretative Abarbeitung des Knotenautomaten (s. Abschnitt 1, Punkt 2) und ersetzen dort die physische Eingabe über Tastatur bzw. Schreibwerk durch die in 6.1.1/6.1.2 beschriebenen Mechanismen der jeweils ausführenden KI bzw. des Python-Interpreters.

6.2 Kanäle K1/K2 (Lochstreifenleser): Eingabe über virtuelle Lochstreifen-Dateien

6.2.1 Trigger und Abgrenzung

Erreicht die interpretative Abarbeitung des Knotenautomaten einen Eingabebefehl ("Eingabe-Halt"), bei dem über die aktive Kanalanwahl ein Lochstreifenleser angefordert wird (Kanal K1 = Leser 1, Kanal K2 = Leser 2, s. Datei "C8205-Interpreter-Lochstreifenleser.md", Abschnitt A5), so wird die Eingabe nicht über die Host-Tastatur (s. 6.1) angefordert, sondern aus einer für den jeweiligen Leser angeforderten Textdatei entnommen, die den "virtuellen Lochstreifen" dieses Lesers repräsentiert.

Für Leser 1 (K1) und Leser 2 (K2) sind stets getrennte Lochstreifendateien zu verwenden; eine für einen der beiden Leser angeforderte Datei wird nicht für den jeweils anderen Leser herangezogen.

Jeder Leser muss immer durch ein Interpreter-Flag definiert werden (Python-Interpreter: `--leser1`/ `--leser2`, als Schalter oder als Zeile der Testdatei, 2b; Claude-Interpreter: entsprechende virtuelle Zuordnung bei der Testdefinition). Fehlt dieses Flag für einen Leser, kann auf dem betreffenden Kanal keine Eingabe erfolgen.

Ist das Flag gesetzt, aber der angegebene Pfad (nur beim Python-Interpreter relevant) existiert nicht, wird dies bereits bei der formalen Vorab-Prüfung (Abschnitt 5) als Fehler erkannt und gemeldet; der Test wird nicht gestartet. Existiert der Pfad, ist aber (aktuell) keine Datei vorhanden bzw. die aktuell geladene Datei zu Ende, während ein Eingabebefehl auf diesen Leser ansteht, wartet der Interpreter still (ohne Meldung) auf das Erscheinen einer neuen Datei (s. 6.2.2) – das ist kein Fehler.

6.2.2 Zeichenentnahme und Nachfordern neuer Dateien

Aus der für den jeweiligen Leser aktuell geladenen Lochstreifendatei wird für jeden von diesem Leser zu interpretierenden Eingabebefehl jeweils ein Zeichen (eine Zeile gemäß 6.2.4) entnommen, fortlaufend in der in der Datei vorliegenden Reihenfolge.

Ist die für einen Leser aktuell geladene Datei zu Ende (keine weitere Zeile mehr vorhanden) und tritt für denselben Leser ein weiterer Eingabebefehl auf, wird für diesen Leser eine neue Lochstreifendatei angefordert.

6.2.3 Wiederverwendung nicht verbrauchter Dateien

Wird eine für einen Leser angeforderte Lochstreifendatei durch die im bisherigen Testverlauf tatsächlich aufgetretenen Eingabebefehle dieses Lesers nicht vollständig "verbraucht" (es sind also noch nicht entnommene Zeilen vorhanden), so bleibt diese Datei für den Leser geladen und wird bei einem im weiteren Testverlauf folgenden Eingabebefehl desselben Lesers weiterverwendet – die Entnahme setzt dabei an der zuletzt entnommenen Stelle fort (s. 6.2.2) –, bis die Datei ihrerseits zu Ende ist oder die Interpretation endet (s. 6.2.5).

6.2.4 Format der virtuellen Lochstreifendateien

Das Format der virtuellen Lochstreifendateien ist binär und an die Datenspuren-Spalten "w v u p 8 Tr 4 2 1" der Tabelle 1 (Datei "C8205-Interpreter-Kodetabellen.md", Abschnitt 1) angelehnt, dargestellt mit den Zeichen "0"/"1" ("1" = Lochung vorhanden, "0" = keine Lochung). Die Transportlochung Tr (ohne Information, s. Datei "C8205-Interpreter-Lochstreifenleser.md", Abschnitt A4) wird dabei nicht dargestellt. Eine Zeile der Datei besteht somit aus den acht 0/1-Werten der Datenspuren w v u p 8 4 2 1 (in dieser Reihenfolge) und repräsentiert jeweils eine Lochungszeile (ein Zeichen) des virtuellen Lochstreifens.

6.2.5 Ende der Interpretation

Geht der Rechner in den Halt-Zustand über (Stopp-Befehl, Fehler-Halt oder auf sonstige Weise), werden alle für den betreffenden Test ggf. geladenen bzw. angebrochenen Lochstreifendateien – für Leser 1 und Leser 2 gleichermaßen – beendet und entfernt.

Eine neue Interpretation eines Tests fordert ihre eigenen Lochstreifendateien für Leser 1 und Leser 2 unabhängig neu an; eine Wiederverwendung von Dateien über das Ende einer Testinterpretation hinaus findet nicht statt.

6.3 Ausgabebefehle

6.3.1 Kanal K2: Schreibwerk-Ausgabe

Kanal und Unabhängigkeit vom Schalter SW

Schreibwerk-Ausgabe erfolgt ausschließlich über Kanal **K2** (s. Datei "C8205-Interpreter-Lochstreifenleser.md", Abschnitt C1) und ist dabei **unabhängig** vom Schalter/Flag SW – dieser betrifft ausschließlich die Eingabeseite (Kanal K0, s. 6.1). Kanal K0 ist reiner Eingabekanal und kommt für Ausgabe nicht in Frage; eine Wartebedingung auf den Schalter SW gibt es bei der Ausgabe nicht.

Ziel der Ausgabe: Datei oder Terminal, gesteuert durch ein Interpreter-Flag

Wird dem Interpreter für die Schreibwerk-Ausgabe eine Datei zugeordnet (Python-Interpreter: Schalter `--sw-ausgabe PFAD` oder gleichlautende Zeile in der Testdatei, s. 2b; Claude-Interpreter: entsprechend virtuell bei der Testdefinition), so geht jede Ausgabe auf K2 in diese Datei "sw-ausgabe.txt". Fehlt diese Zuordnung, geht jede Ausgabe auf K2 stattdessen aufs Terminal, dort in Dunkelgelb dargestellt. Mit Zuordnung geht die Ausgabe in die Datei, ohne Zuordnung aufs Terminal; beides schließt sich gegenseitig aus. Unabhängig davon kann der Python-Interpreter mit dem Kommandozeilenschalter `--SW-TERMINAL` aufgerufen werden (Ergänzung 10.10.2026): Dann wird neben dem Terminal ein Ausgabefenster „SW Ausgabe“ geöffnet, in dem **jede** Ausgabe auf K2 zusätzlich erscheint, gleichgültig ob eine Datei zugeordnet ist.

Schalter `--SW-TERMINAL`**:** nicht angegeben bedeutet AUS (kein Fenster), angegeben bedeutet EIN. Er wird bei der Durchführung des Tests ausgewertet und gilt für alle Tests eines Aufrufs; er ist nur auf der Kommandozeile zulässig, **nicht** als Zeile in der Testdatei (2b). Das Fenster enthält nur Text (130 Zeichen × 36 Zeilen), trägt die Überschrift „SW Ausgabe“, ist verschiebbar und erscheint standardmäßig rechts unten auf dem Desktop. Das Fenster bleibt auch nach dem Ende des Aufrufs **immer offen**, bis es mit Klick auf X geschlossen wird. Die Wirkung von `--sw-ausgabe` (Datei, sonst Terminal) bleibt unverändert. Mit `--SW-TERMINAL` erscheint außerdem jede angenommene Eingabe über K0 (Pult-Tastatur T oder Schreibwerk SW, 6.1) als Echo in Dunkelgelb im selben Fenster; die Ausgaben des Rechners stehen dort in der Standardfarbe. Die Eingaben der Lochbandleser (K1, K2) werden nicht ausgegeben.

> **Hinweis:** Das Fenster „SW Ausgabe“ schließt sich nie von selbst; es bleibt offen, bis es mit Klick auf X geschlossen wird. Text im Fenster lässt sich mit der Maus markieren und in die Zwischenablage kopieren (Loslassen der Maustaste, Strg+C, Strg+Einfg, Kontextmenü; Windows und Linux; das Kopierverfahren wählt der Schalter `--zwischenablage auto|tk|system`, Python-Interpreter, Abschnitt 5.1).

Anlegen und Fortführen der Datei

Die Datei "sw-ausgabe.txt" wird für einen Test höchstens einmal angelegt. Sie wird nicht angelegt, wenn der betreffende Test keine Ausgabe auf K2 vornimmt. Erfolgen im Testverlauf mehrere Ausgaben auf K2, wird stets dieselbe, bereits angelegte Datei fortgeführt (Ausgabe wird angehängt, nicht überschrieben).

Zeichenweise Ausgabe und Zeilenende

Die Ausgabe erfolgt zeichenweise. Das Zeichen WRZL (Wagenrücklauf Zeilenvorschub) kennzeichnet dabei das Erreichen des Zeilenendes: Bei Ausgabe von WRZL wird die aktuelle Zeile der Datei abgeschlossen, ggf. nachfolgende Ausgaben beginnen auf einer neuen Zeile. Mehrere unmittelbar aufeinanderfolgende WRZL-Ausgaben erzeugen entsprechend mehrere Leerzeilen.

Bereitstellung am Testende

Eine im Testverlauf ggf. erzeugte Datei "sw-ausgabe.txt" wird am Ende des Tests zum Download bereitgestellt – dies ist nur beim Claude-Interpreter erforderlich; beim Python-Interpreter liegt die Datei (sofern per `--sw-ausgabe` zugeordnet) ohnehin bereits im lokalen Dateisystem.

6.3.2 Kanal K1 (Lochstreifenstanzer): Ausgabe in die Datei "stanzer-ausgabe.txt"

Flag-Pflicht, kein Terminal-Ausweg

Für Ausgabebefehle auf Kanal K1 (Lochstreifenstanzer, s. Datei "C8205-Interpreter-Lochstreifenleser.md", Abschnitt B5) ist das Flag `--stanzer1` (Python-Interpreter, Schalter oder Zeile der Testdatei, 2b) bzw. die entsprechende virtuelle Zuordnung (Claude-Interpreter) erforderlich. Fehlt diese Zuordnung, kann auf K1 **keine** Ausgabe erfolgen – anders als bei der Schreibwerk-Ausgabe (6.3.1) gibt es für den Stanzer **keinen** Terminal-Ausweg: ohne `--stanzer1` unterbleibt die Ausgabe schlicht, es erscheint auch nichts am Terminal.

Ist `--stanzer1` gesetzt, gilt Abschnitt 6.3.1 sinngemäß entsprechend: höchstens eine Datei je Test, Anlegen nur bei tatsächlich erfolgter Ausgabe, Fortführung derselben Datei bei mehreren Ausgaben, Bereitstellung zum Download am Testende – auch hier nur beim Claude-Interpreter erforderlich, da die Datei beim Python-Interpreter ohnehin lokal vorliegt. Eine Wartebedingung wie früher für die Schreibwerk-Ausgabe angenommen ist hier (und nach Korrektur von 6.3.1 auch dort) nicht vorgesehen.

Abweichend von 6.3.1 werden jedoch ALLE Zeichen jeweils in einer eigenen Zeile im Binärformat ausgegeben, gemäß den Datenspuren-Spalten "w v u p 8 Tr 4 2 1" der Tabelle 1 (Datei "C8205-Interpreter-Kodetabellen.md", Abschnitt 1; "1" = Lochung vorhanden, "0" = keine Lochung); die Transportlochung Tr wird dabei, wie schon bei den virtuellen Lochstreifen-Eingabedateien (s. 6.2.4), nicht dargestellt. Eine Zeile besteht somit aus den acht 0/1-Werten der Datenspuren w v u p 8 4 2 1 (in dieser Reihenfolge) und repräsentiert jeweils ein ausgegebenes

Zeichen. Das Konzept "Zeilenende durch WRZL" (s. 6.3.1) ist auf diese Datei nicht anwendbar, da hier ohnehin jedes Zeichen bereits eine eigene Zeile bildet.

7. Diagnoseausgabe bei Befehl 164 (Leerbefehl) mit Testmarken #DOK/#SPEI

Die im Folgenden beschriebene Diagnoseausgabe ist, wie die zugrundeliegenden Testmarken #DOK-xxxx und #SPEI-xxxx (s. Abschnitt 2), eine von der realen Hardware abweichende, zusätzliche Definition, die ausschließlich der besseren Durchführung von Tests mit dem Interpreter dient und keine Entsprechung in der physischen Elektronik des C 8205 hat.

7.1 Auslösebedingung

Tritt bei der Interpretation der im virtuellen Trommelspeicher definierten Befehlsfolge der Befehl 164 (oktal; Leerbefehl, B=1/A=6/Z=4, s. Befehlsliste, Abschnitt 4.3, Organisationsbefehle 2. Art, A=6 / Z=4–7) mit einer Adresse a auf, für die in der Trommeldatei eine Testmarke #DOK-xxxx mit xxxx=a definiert ist, so wird zusätzlich zur regulären Abarbeitung die im Folgenden beschriebene Diagnoseausgabe vorgenommen.

7.2 Ablauf

1. Der Befehl 164 wird zunächst interpretativ genau so abgearbeitet, wie es der Knotenautomat für diesen Befehl vorgibt; die Ausführung selbst erfährt keine Sonderbehandlung.
2. Danach wird in die Protokolldatei des Tests (s. Abschnitt 8) geschrieben:
   - die Testmarke #DOK-xxxx mit dem in der Trommeldatei zugehörig angegebenen Text,
   - der Inhalt der Register <BZ> und <AC>,
   - die Werte der Flags FE, FÜ, BS, US, E, A (Ü steht hier für Flag FÜ, s. Flagtabelle in Datei "C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md").
3. Ist zu demselben xxxx zusätzlich eine Testmarke #SPEI-xxxx ssss-eeee in der Trommeldatei definiert, werden danach zusätzlich alle virtuellen Trommelinhalte der Adressen ssss bis eeee zeilenweise in die Protokolldatei geschrieben, jeweils Adresse und Inhalt in Oktalziffern (Zeilenformat entsprechend Abschnitt 2: "Adresse: Wert").
4. Ist zu demselben xxxx zusätzlich eine Testmarke #SPUR-xxxx aaaa definiert, werden danach (nach allen #SPEI-xxxx-Ausgaben) alle 32 Zellen der Spur, in der aaaa liegt, zeilenweise ausgegeben, und zwar in physischer Zugriffsreihenfolge der Trommel (s. Abschnitt 2); Zeilenformat wie bei #SPEI-xxxx.
5. Danach wird die Interpretation gemäß Knotenautomat regulär fortgesetzt.

Diese Bildschirmausgabe der Diagnoseinformationen erfolgt unabhängig von der in #PROT STUFE=n (s. Abschnitt 2) festgelegten Protokollierungsstufe stets zusätzlich auf dem Bildschirm; sie ist von der in Abschnitt 8 beschriebenen, stufenabhängigen Protokollierung des gesamten Knotenautomaten-Durchlaufs zu unterscheiden. Bei STUFE=2 wird sie außerdem, wie in Schritt 2 oben beschrieben, zusätzlich in die Protokolldatei geschrieben.

8. Protokolldatei je Test

Bei Protokollierungsstufe STUFE=2 (s. #PROT STUFE=n, Abschnitt 2) wird für den Test eine eigene Protokolldatei ("protokoll.txt") angelegt. Sie ist von den in Abschnitt 6.3 beschriebenen Dateien "sw-ausgabe.txt" und "stanzer-ausgabe.txt" zu unterscheiden: Während jene nur bei tatsächlich erfolgter Ausgabe auf dem jeweiligen Kanal angelegt werden, wird die Protokolldatei bei STUFE=2 für den gesamten Test angelegt, unabhängig davon, ob Ausgabebefehle auftreten. Bei STUFE=0 und STUFE=1 entsteht keine Protokolldatei.

8.1 Inhalt

Bei STUFE=2 enthält die Protokolldatei, in der zeitlichen Reihenfolge ihres Auftretens während des Tests:

- die Protokollierung der Ausgangsbedingungen: die vom Test übernommenen Trommelzeilen (Abschnitt 2; nicht der ganze Trommelinhalt, Abschnitt 2c), die Testmarken #AC= und #BZ= (falls angegeben, mit den Anfangswerten von AC und BZ), die Testmarke #TROMMEL= (falls angegeben, Abschnitt 2g) sowie die Tastenfolge zur Definition des Ausgangszustands mit ihrer Wirkung auf Register und Flags (Abschnitt 3),
- die Protokollierung der Zustände des Knotenautomaten: die beim Durchlauf des Knotengraphen ab Konnektor B (Datei "C8205-Interpreter-Eingänge_der_Knoten.md") jeweils erreichten Knoten in der durchlaufenen Reihenfolge,

- die angeforderten Debug-Ausgaben gemäß Abschnitt 7 (`#DOK-xxxx`-Ausgaben mitsamt Registern und Flags sowie ggf. zugehörige `#SPEI-xxxx`- und `#SPUR-xxxx`-Speicherauszüge),
- abschließend, sofern für den Test ein `Prüfungen:`-Abschnitt angegeben ist (Abschnitt 3a), die Ergebnisprüfung: je geprüftem Register/Flag bzw. je geprüfter Speicherzelle Soll- und Ist-Wert, sowie das Gesamtergebnis `Test wahr` / `Test falsch`. Dazu gehören auch die Dateiprüfungen nach Abschnitt 3b (Vergleich der Stanzer- bzw. Schreibwerk-Ausgabedatei mit der Soll-Datei). Bei `Prüfungen: False` folgt die Vermerkzeile nach Abschnitt 3a.5.

8.2 Bildschirmausgabe der Zustände des Knotenautomaten

Bei STUFE=1 wird die Protokollierung der Zustände des Knotenautomaten auf dem Bildschirm ausgegeben, in möglichst einzeiliger, knapper Form (z. B. eine Zeile je durchlaufenem Knoten); da bei STUFE=1 keine Protokolldatei entsteht (s. o.), ist dies dort die einzige Aufzeichnung dieser Zustände. Bei STUFE=2 entfällt diese Bildschirmausgabe; die Zustände werden stattdessen gemäß 8.1 in die Protokolldatei geschrieben. Bei STUFE=0 entfällt sie ebenfalls, ohne Ersatz durch eine Protokolldatei. Die Diagnoseausgaben zu Befehl 164 (Abschnitt 7) sind von dieser stufenabhängigen Regelung nicht erfasst; sie erscheinen stufenunabhängig immer auf dem Bildschirm.

8.3 Bereitstellung am Testende

Die Protokolldatei wird, sofern sie angelegt wurde (STUFE=2, s. o.), am Ende des Tests zum Download bereitgestellt.

9. Ausblick

Dieses Dokument beschreibt die Durchführung eines Tests: den Aufbau der Testdatei (Abschnitt 2 bis 2g), drei durchgeführte Beispiele (2h bis 2j) und die Übersicht der Lademethoden mit ihrem Angang am Assemblerquelltext (2k), die Tastenfolge zum Aufstellen des Ausgangszustands (Abschnitt 3), die Ergebnisprüfung (3a) und die Dateiprüfungen (3b), weitere Tastensequenzen (Abschnitt 4), die formale Gültigkeitsprüfung (Abschnitt 5), die Zeichenein- und -ausgabe (Abschnitt 6), die Diagnoseausgabe (Abschnitt 7) und die Protokolldatei (Abschnitt 8). Der eigentliche Durchlauf durch den Knotengraphen ab Konnektor B ist in der Datei "C8205-Interpreter-Eingänge_der_Knoten.md" beschrieben.